



Cloud P2P Application for Image Sharing

Final Report

Fall 2025

CSCE4411- Fund. of Distributed Systems

The American University in Cairo

Group 706

Michael Reda	900203291
Knzy Elmasry	900202766
Freddy Amgad	900203088
Andrew Aziz	900213227

Submitted to:

Dr. Amr El Kadi

1. Project Overview.....	3
1.1 Introduction.....	3
1.2 Project Description.....	3
1.3 Project Requirements.....	4
1.3.1 Service Side.....	4
1.3.2 Client Side.....	5
2. Design Choices.....	6
2.1 Leader Election Architecture.....	6
2.1.1 Election Algorithm Choice (Raft).....	6
2.1.2 Handling the "Single Server" Edge Case.....	6
2.1.3 Timeout-Based Peer Exclusion.....	6
2.1.4 Fault Tolerance & Single-Node Survivability.....	7
2.2 System Load Balancing.....	7
2.2.1 The "Elect-and-Execute" Architecture.....	7
2.2.2 The Metric-Based Decision Algorithm.....	8
2.2.3 The Feedback Loop.....	8
2.2.4 Justification of Design.....	9
2.3 Directory of Service (DoS).....	9
2.3.1 Active Server vs. Passive File.....	9
2.3.2 Liveness Detection (The Monitor).....	10
2.3.3 Directory Fault Tolerance (Replication).....	10
2.3.4 Persistence.....	10
2.3.5 Summary of Design Decisions & Justifications.....	10
2.4 Defining Client Request Types.....	11
2.4.1 Request Formats and Serialization Mechanisms.....	11
2.4.1.1 JSON-based Requests (The Control Plane).....	11
2.4.2 Image Transfer Request (The Data Plane).....	13
2.5 Data Marshalling and Serialization.....	14
2.5.1 Serialization and Communication in the Cloud P2P Environment.....	14
2.5.1.1 Network Communication (JSON).....	14
2.5.1.2 Steganographic Embedding (Bincode).....	14
2.5.2 Comparison of Packet Sizes and Efficiency.....	15
2.5.3 The Unified Payload Structure.....	15
2.6 Interprocess Communication (IPC).....	15
2.6.2 Client-to-Server Communication (The Discovery Layer).....	16
2.6.3 Client-to-Client Communication (The P2P Layer).....	16
2.7 Encryption.....	16
2.7.1 Image Encoding (Client-Server).....	17
2.7.2 Access Rights Encoding (Client-Client).....	17
2.8 Code Structure & System Component Map.....	17

2.8.1 Core Shared Library (src/lib/)	18
2.8.2 Service Executables (The Backend Nodes)	18
2.8.3 The Client Application (Frontend & Bridge)	19
2.8.4 Architectural Data Flow	19
3. Detailed Use Cases	20
3.1 Registration and Network Entry	21
3.2 Encrypting an Image (Phase 1 Integration)	24
3.3 Peer Discovery & Thumbnail Caching	27
3.4 Requesting Views (Online Scenario)	28
3.5 Requesting Views with Failure Handling (Offline)	29
3.6 Viewing the Image (Client-Side)	31
3.7 Client Controls Access Rights (Revocation) online	34
3.8 Client Controls Access Rights (Revocation) offline	36
4. System Performance (From Phase 1 Report)	38
4.1 The Stress Testing Methodology	38
4.2 Analysis of System Success rate Under Load Balancing and Fault Conditions	38
4.4 Analysis of System Average Response Time (Load Balancing Enabled, No Fault Simulation)	40
4.5 Analysis of System Average Response Time (No Load Balancing)	41
4.6 Conclusion	42
5. Future Improvements & System Evolution	43
6. References	44

1. Project Overview

1.1 Introduction

In the modern landscape of distributed systems, the reliance on centralized servers often creates bottlenecks in performance and single points of failure. Our project, "Cloud P2P Environment for Controlled Sharing of Images," addresses these challenges by implementing a hybrid architecture that combines the robustness of a consensus-based cloud cluster with the scalability of a Peer-to-Peer (P2P) network.

While Phase 1 of this project focused on establishing a fault-tolerant cloud cluster capable of performing computationally expensive tasks specifically, image encryption using Steganography and the Raft consensus algorithm, Phase 2 expands this ecosystem to the edge. In this phase, we have developed a decentralized sharing mechanism where clients communicate directly with one another to exchange data, drastically reducing the bandwidth load on the central cloud.

This report details the design, implementation, and analysis of this Phase 2 evolution. It specifically covers the introduction of a Directory of Service (DoS) for peer discovery, a custom P2P Application Layer Protocol for image transfer, and a "Best Effort" Offline Messaging System to handle the volatile nature of peer availability.

1.2 Project Description

The core objective of this project was to design and implement a **Hybrid Distributed System** that merges the security of centralized processing with the efficiency of decentralized distribution. Traditional image sharing platforms (like Google Photos or Dropbox) rely heavily on central servers to store and serve data. While simpler to implement, this architecture creates significant bottlenecks: bandwidth saturation at the server, single points of failure, and high latency for geographically distant users.

Our solution, the **Cloud P2P Environment**, addresses these limitations by decoupling the system into two distinct operational planes:

1. **The Computational Plane (The "Cloud"):** This layer is responsible for the CPU-intensive task of **Steganographic Encryption**. We utilized the Raft consensus algorithm (implemented in Phase 1) to create a fault-tolerant cluster of servers. These servers receive raw images, embed complex access-rights metadata (permissions) into the pixel bits, and return the secure asset to the user. This ensures that the "heavy lifting" is handled by a dedicated, load-balanced cluster rather than user devices.
2. **The Distribution Plane (The "P2P Network"):** Once an image is secured, the storage and sharing burden shifts entirely to the edge. In Phase 2, we transformed the client application from a passive requester into an active **Peer Node**. Each client now runs a background TCP listener (using the Rust `tokio` runtime), allowing it to serve images directly to other authenticated users.

The distinguishing feature of our architecture is its focus on "**Controlled Sharing.**" Unlike standard file-sharing networks (like BitTorrent) where data is public, our system enforces strict access control. Even though Alice sends a file directly to Bob, she retains the ability to revoke his access remotely. If Bob is offline during the revocation, our architecture utilizes a "**Best Effort**" **Store-and-Forward** mechanism within the Directory Service to ensure the security policy is eventually applied.

1.3 Project Requirements

For the sake of clarity, the system environment is divided into distinct components. The requirements below outline the functionalities, communication protocols, and objectives of each component.

1.3.1 Service Side

Cloud of Servers (The Computational Cluster)

1. Three interconnected servers communicate via **TCP** to form a reliable consensus cluster.
2. The cloud utilizes the **Raft Consensus Algorithm** to elect a Leader, ensuring coordination across the distributed nodes.
3. The Leader utilizes the election mechanism for **Load Balancing**, delegating computationally intensive encryption tasks to Follower nodes based on CPU availability.
4. The cloud is **Failure Tolerant**; if a Follower fails, the Leader re-assigns work. If the Leader fails, a new election is triggered immediately to maintain system availability.
5. Each server implements **LSB Steganography** to embed access rights (permissions) directly into the image pixels.
6. The servers are stateless regarding image storage; they process raw images and return the encrypted asset to the user, ensuring no centralized data hoarding.

Directory of Service (DoS)

1. The DoS acts as the network's dynamic lookup table, registering users and mapping their logical Usernames to physical addresses (IP Address & Port Number).
2. The DoS actively monitors user availability via **Periodic Signals**. If a client stops sending these messages, the DoS updates their status to **Offline** in real-time.
3. The DoS serves as a "**Best Effort**" **Message Broker**. It receives access control modifications (e.g., Revoke Access) for users who are currently offline and enforces them by delivering the message once the target user reconnects.
4. The DoS is designed for **Fault Tolerance** via having 3 directories of service running, so if the primary directory node fails, a backup node retains the user mapping to prevent network partition.
5. The DoS persists its state to disk (JSON storage) to survive total system power failures.

1.3.2 Client Side

Client Application (User Interface)

1. Each client acts as an owner of images and can submit them to the Cloud for encryption.
2. The application provides a Command Line Interface (CLI) for users to Request, Share, and Revoke access to images.
3. The client allows the user to specify a precise "View Quota" (e.g., 5 views) when sharing an image with a peer.
4. The client provides visual feedback on the status of peers (Online/Offline) by querying the DoS.

Client Middleware (P2P Logic)

1. The middleware transforms the client into a **Dual-Stack Node**: it runs a main thread for user commands and a background `tokio` task to listen for incoming TCP connections.
2. The middleware handles **Peer-to-Peer (P2P) File Transfer**, allowing the direct streaming of encrypted image data between clients without routing through the cloud.
3. The middleware enforces **Local Security Policies**: before sending an image to a requester, it verifies that the requester has valid access rights.
4. The middleware manages **Digital Rights Management (DRM)** on the receiver's side by automatically decrementing the "Remaining Views" counter after the image is closed.
5. The middleware enforces "Access Denied" logic: if the view quota reaches zero, the middleware prevents the decryption of the hidden pixels and displays a default placeholder image instead.
6. The middleware implements a **Store-and-Forward Client**, automatically routing control messages (like permission updates) to the Directory Service if a direct P2P connection to a peer fails.

2. Design Choices

2.1 Leader Election Architecture

2.1.1 Election Algorithm Choice (Raft)

Referenced Code: `raft.rs`

For the coordination of our Cloud Cluster, we implemented the **Raft Consensus Algorithm**. We selected Raft over Paxos because of its strong "Leader-Follower" model, which centralizes the responsibility of handling client requests and simplifies the consistency logic.

The Quorum Requirement: Standard Raft requires a majority ($N/2 + 1$) to elect a leader. In our 3-node cluster, this means a candidate must receive at least 2 votes (itself + one peer).

- ❖ **Normal Operation:** If 3 nodes are running, a Candidate needs 2 votes.
- ❖ **Partial Failure:** If 1 node crashes, the remaining 2 can still form a quorum and elect a leader.
- ❖ **Critical Failure:** If 2 nodes crash, leaving only 1 node alive, the standard Raft algorithm intentionally halts to prevent "Split Brain" (where two isolated nodes both think they are leaders).

2.1.2 Handling the "Single Server" Edge Case

Referenced Code: `raft.rs - start_election`

A unique challenge in our implementation was handling the scenario where only one server is running (e.g., during initial boot or catastrophic failure).

- ❖ **The Logic:** In our `raft.rs`, we check the `peers` configuration list size.
 - `if peers.len() == 0`: The node detects it is in "Single Mode" and immediately promotes itself to Leader.
 - `if peers.len() > 0`: The node enforces the strict Quorum rule (requiring 2 votes).
- ❖ **Justification:** This hybrid approach allows for easy testing and development (running just one server) while enforcing strict consistency when the full cluster is deployed.

2.1.3 Timeout-Based Peer Exclusion

To prevent system latency when a node becomes unresponsive, we implement the Timeout-Based **Peer Exclusion** mechanism during the metric gathering phase.

The Algorithm:

- ❖ **Parallel Querying:** When a request arrives, the receiving node queries all peers for their load metrics in parallel.
- ❖ **Strict Deadlines:** We enforce a strict timeout on these internal queries.
- ❖ **Exclusion:** If a peer fails to respond within this window, it is immediately marked as "Unreachable" and excluded from the election pool for the current request.

- ❖ **Result:** This prevents the system from hanging or waiting for a failed node. Unlike the original model which required a lengthy re-election phase to recognize a failure, the per-request model detects and isolates dead nodes instantly during the transaction itself.

2.1.4 Fault Tolerance & Single-Node Survivability

Our implementation ensures high availability through **Quorum Relaxation** and **Stateless Failover**, ensuring the system functions even if only one server remains active.

- ❖ **Stateless Redundancy:** Since there is no persistent "Leader" to crash, the failure of any specific node does not halt the system. Client middleware simply routes the *next* request to any surviving node, which will then win the per-request election by default.
- ❖ **Single-Node Mode:** Originally, the system required a majority (quorum) to process requests. We removed this constraint; if a node detects that all peers are offline (via the timeout mechanism in 2.1.3), it automatically switches to **Single-Node Mode** and processes requests locally.
- ❖ **Recovery via History:** If a node crashes *during* processing, the **Synchronized Request History** ensures that when it (or a peer) comes back online, the pending status is reconciled, making the failure transparent to the user without manual intervention.

2.2 System Load Balancing

Referenced Code: `server.rs`

In Phase 1, we identified that **Steganographic Encryption is a CPU-bound operation**. A single high-resolution image encryption request can spike a CPU core's usage for several seconds. To prevent any single node from becoming a bottleneck, we replaced the traditional centralized "Dispatcher" model with a decentralized **Per-Request "Elect-and-Execute" Strategy**.

2.2.1 The "Elect-and-Execute" Architecture

Unlike standard web server architectures where a central load balancer accepts traffic and delegates it (the "Dispatcher" model), our system utilizes a decentralized decision-making process for every incoming task to minimize network latency.

- ❖ **Multicast Discovery:** Instead of connecting to a static leader, the Client's middleware multicasts the request metadata to all available server peers.
- ❖ **Dynamic Assessment:** Upon receiving the metadata, every server node instantly evaluates its own current capability using real-time metrics, specifically **CPU Load** (critical for encryption) and **Active Connection Count**.
- ❖ **Distributed Election:** The nodes engage in a rapid, per-request election protocol. The node with the lowest calculated "Load Score" is elected as the "Temporary Leader" for this specific request.
- ❖ **Local Execution (No Forwarding):** Crucially, the elected node **processes the request locally**. This eliminates the "double-hop" inefficiency found in delegation architectures (where a Leader accepts data only to forward it to a Follower). By ensuring the node that *wins* the election is the

one that *does* the work, we significantly reduce internal cluster bandwidth usage and serialization overhead.

2.2.2 The Metric-Based Decision Algorithm

We rejected simple Round-Robin load balancing because it is blind to the actual state of the servers. Instead, we implemented a **Weighted Metric Score**.

The Load Score Formula: Defined in `lib.rs`, every node calculates a score between 0.0 (Idle) and 1.0 (Overloaded).

$$Score = (W_1 * CPU) + (W_2 * Connections) + (W_3 * ResponseTime)$$

Code Insight: **The Metric Calculation.** The following code from `lib.rs` demonstrates how we normalize these disparate metrics into a single comparable value:

// From `lib.rs`

```
impl ServerMetrics {
    /// Calculate load score for this server (lower is better)
    /// This score is used to determine which server should handle incoming work
    pub fn calculate_load_score(&self) -> f32 {
        // Weighted scoring formula - adjust these weights based on your priorities
        let cpu_weight = 0.4;
        let connection_weight = 0.4;
        let response_weight = 0.2;

        // Normalize values to 0.0-1.0 range
        let normalized_cpu = self.cpu_load / 100.0;
        let normalized_connections = (self.active_connections as f32) / 50.0; // assume max 50 connections
        let normalized_response = (self.avg_response_time_ms as f32) / 10000.0; // normalize to 10 seconds

        // Calculate weighted sum
        cpu_weight * normalized_cpu +
        connection_weight * normalized_connections +
        response_weight * normalized_response
    }
}
```

2.2.3 The Feedback Loop

To ensure the Leader has up-to-date information, we piggybacked the metric reporting onto the Raft consensus messages.

- ❖ **Mechanism:** When a Follower responds to an update message (AppendEntries RPC), it includes its latest ServerMetrics.
- ❖ **Latency:** Since system updates occur every 50ms, the Leader's view of the cluster state is never more than 50ms old.

- ❖ **Benefit:** This "Piggybacking" technique avoids the need for opening separate TCP connections just for status updates, reducing network overhead.

2.2.4 Justification of Design

This architecture allows our system to handle "**Bursty**" Traffic. If 10 users submit large images simultaneously:

1. The Leader takes Request 1.
2. The Leader sees its Load Score rise.
3. The Leader forwards Request 2 to Follower A.
4. The Leader forwards Request 3 to Follower B.

Result: The heavy encryption work happens in parallel across 3 machines, while the Leader remains responsive to new connection attempts.

2.3 Directory of Service (DoS)

Referenced Code: `directory_service.rs`, `directory_server.rs`

The Directory of Service acts as the authoritative "Phonebook" for the P2P network. Its primary responsibility is to map logical Usernames (e.g., "Alice") to physical network addresses (e.g., `10.40.7.5:8081`) while maintaining the real-time status of each peer.

2.3.1 Active Server vs. Passive File

The project requirements initially suggested using a "Shared File" (like Google Drive) to act as a simple directory. However, we identified a critical flaw in that approach: **Staleness**. A static file cannot actively detect if a user has crashed; it only records the last write operation. This would lead to peers attempting to connect to dead IP addresses.

Our Design Choice:

We implemented a custom **Active TCP Directory Server**.

Structure:

The directory maintains an in-memory `HashMap<String, UserEntry>` protected by a `Tokio RwLock` to allow high-concurrency reads.

The User Entry:

```
pub struct UserEntry {
    pub username: String,
    pub p2p_address: String,
    pub last_message: SystemTime,
    pub status: UserStatus,
    pub shared_images: Vec<ImageInfo>,
}
```

// From `directory_service.rs` Where the `last_message` is the “liveness” timestamp and how we determine a user’s `status` offline or online.

2.3.2 Liveness Detection (The Monitor)

To ensure the directory is always accurate, we implemented a background monitoring task.

- ❖ **Mechanism:** Every client sends a periodic message to the Directory every 5 seconds.
- ❖ **Detection:** The Directory Server runs a loop that scans all registered users every second.
 - **Logic:** if the last message the user sent passed a certain threshold in terms of timeout, then user is offline
 - **Action:** The user's status is automatically downgraded to `Offline`.
- ❖ **Benefit:** This provides a "Liveness Guarantee." When Peer A queries for Peer B, they receive the real-time status, preventing wasted connection attempts.

2.3.3 Directory Fault Tolerance (Replication)

Recognizing that a centralized Directory Server creates a Single Point of Failure (SPOF), we implemented **Primary-Backup Replication**.

- ❖ **Implementation:** We run multiple instances of the Directory Server (e.g., `dir1` on port 9000, `dir2` on port 9001).
- ❖ **Synchronization:** When the Primary Directory receives a `Register` or `UpdateStatus` request, it broadcasts a `SyncState` message to its configured peer servers.
- ❖ **Failover:** Clients are configured with a list of directory servers. If the connection to the Primary fails (timeout), the client automatically attempts to connect to the Secondary.

2.3.4 Persistence

To survive a total system shutdown (e.g., power outage affecting all nodes), the Directory Service implements **Disk Persistence**.

- ❖ **Action:** On every state change, the `user's` HashMap is serialized to JSON and written to `directory_state.json`.
- ❖ **Recovery:** Upon restart, the Directory Server reads this file to restore the `Username -> IP` mapping, ensuring the network topology is not lost.

2.3.5 Summary of Design Decisions & Justifications

To ensure the Directory Service met the requirements of **availability** and **liveness**, we made several critical engineering decisions. The table below summarizes the alternatives considered and the justification for our final architecture.

Design Decision	Alternative Considered	Selected Approach	Justification
Storage Medium	Shared Network File (e.g., Google Drive)	Active Memory (HashMap) + Disk Persistence	A shared file suffers from race conditions (two users writing simultaneously) and staleness (cannot detect crashes). An in-memory HashMap is O(1) for lookups, while the background thread handles the "liveness" logic that a static file cannot.
Protocol	UDP (User Datagram Protocol)	TCP (Transmission Control Protocol)	While UDP is faster, it is unreliable. If a packet is dropped, the system loses track of a user. We chose TCP to guarantee that critical topology updates are delivered and ordered correctly.
Consistency	Strong Consistency (Raft)	Eventual Consistency (Primary-Backup)	Implementing Raft for the Directory would have introduced unnecessary latency for simple lookups. We chose a simpler Primary-Backup replication model because "Phonebook" data does not require the strict transactional safety of a banking system, allowing for faster read speeds.
Offline Logic	Reject requests for offline users	Store-and-Forward (Message Queue)	Simply rejecting requests would fail the "Best Effort" requirement. By acting as a temporary message broker, the Directory Service ensures that permission revocations are eventually enforced, closing a potential security loophole.

2.4 Defining Client Request Types

Referenced Code: `p2p_protocol.rs`, `client.rs`

A critical aspect of our architecture was defining a robust communication standard that could handle both high-frequency control messages and high-bandwidth data transfers. To achieve this, we categorized client requests into two distinct planes: the **Control Plane** (Logic) and the **Data Plane** (Payload).

2.4.1 Request Formats and Serialization Mechanisms

2.4.1.1 JSON-based Requests (The Control Plane)

For all logic-based interactions, where ensuring correct state is more important than raw speed, we utilized **JSON** serialization via the `serde_json` crate.

- ❖ **Usage:** Directory Handshakes (`Register`, `QueryPeers`), Raft Consensus RPCs (`RequestVote`), and P2P Coordination (`UpdatePermissions`).
- ❖ **Justification:** JSON is human-readable. This was invaluable for debugging distributed race conditions. We could simply tail the server logs and see: `[INFO] Received: {"type": "ImageRequest", "user": "Alice", "image": "vacation.png"}`.

The Structure: All P2P control messages are encapsulated in the `P2PMessage` enum:

Rust

// From `p2p_protocol.rs`

```
pub enum P2PMessage {  
    /// Request to view an image with a specific number of views  
    ImageRequest {  
        requesting_user: String,  
        image_id: String,  
        requested_views: u32,  
    },  
}
```

```

    /// Request to update permissions for an already-shared image
    UpdatePermissions {
        owner: String,
        image_id: String,
        username: String,
        new_quota: u32,
    },

    /// Response to permission update request
    UpdatePermissionsResponse {
        success: bool,
        message: String,
    },

    /// Deliver image to requester after owner accepts (push model)
    DeliverImage {
        from_owner: String,
        image_id: String,
        requested_views: u32,
        encrypted_image: Vec<u8>, // The actual image data with embedded permissions
    },

    /// Remote permission update: Owner asks requester to update their local copy's permissions
    RemoteUpdatePermissions {
        from_owner: String,
        image_id: String,
        for_user: String,
        new_quota: u32,
    },

```

// ...

2.4.2 Image Transfer Request (The Data Plane)

For the actual transmission of the encrypted image, using JSON would be inefficient. Converting binary data to a Base64 string increases file size by approximately 33%. For a 15MB image, this would result in 20MB of network traffic.

- ❖ **Our Solution:** We switched to **Raw Byte Streams** for the payload transfer.
- ❖ **The Protocol:**
 - **Header:** Send a 4-byte integer representing the file size (u32).
 - **Body:** Stream the raw bytes of the PNG file immediately after.

❖ **Code Implementation:**

```
// From p2p_protocol.rs
// Send response
let response_json = serde_json::to_string(&response)?;
let response_bytes = response_json.as_bytes();

stream.write_u32(response_bytes.len() as u32).await?;
stream.write_all(response_bytes).await?;
```

- ❖ **Benefit:** This "Zero-Overhead" approach ensures we fully utilize the available network bandwidth, providing the fastest possible download speeds for peers.

2.5 Data Marshalling and Serialization

Referenced Code: `lib.rs`, `lsb.rs`, `server_gui.rs`

In a distributed system, data marshalling is the process of transforming memory objects into a format suitable for storage or transmission and it is a critical performance factor. For our Cloud P2P environment, we adopted a **Hybrid Serialization Strategy** to balance the conflicting requirements of **Human Readability** (for debugging) and **Bitwise Efficiency** (for steganography).

2.5.1 Serialization and Communication in the Cloud P2P Environment

We distinctively separated our serialization logic into two domains:

2.5.1.1 Network Communication (JSON)

For all RPCs (Remote Procedure Calls) between Raft nodes and all handshake messages between Clients and the Directory, we utilized **JSON** (`serde_json`).

Justification: The network bandwidth consumed by control messages is negligible compared to image data. Using JSON allowed us to inspect traffic using standard tools (like Wireshark or simple log files) to verify the system state.

- ❖ *Example:* `{"type": "RequestVote", "term": 4, "candidate_id": "server1"}` is immediately understandable, whereas a binary blob would require a custom decoder to debug.

2.5.1.2 Steganographic Embedding (Bincode)

For the data hidden *inside* the image pixels, we switched to **Bincode** (a compact binary serialization format).

- ❖ **The Constraint:** Steganography has a strict "Capacity Limit." Hiding data in the Least Significant Bit (LSB) of a 1000x1000 image gives us exactly 3,000,000 bits (~375KB) of storage.
- ❖ **The Solution:** We must fit the `ImagePermissions` struct and the `UnifiedPlaceholder` image into this space.

- **JSON Overhead:** A JSON string `{"owner": "Alice", "views": 5}` takes ~35 bytes.
- **Bincode Efficiency:** The same struct in Bincode takes ~12 bytes.
- ❖ **Impact:** Using Bincode allowed us to embed higher-quality placeholder images and more complex permission lists without exceeding the pixel capacity of the cover image.

2.5.2 Comparison of Packet Sizes and Efficiency

To validate this design choice, we conducted a comparison of the payload sizes for a standard `ImagePermissions` object containing 10 users.

Format	Serialized Size (Bytes)	Human Readable?	Parse Speed	Use Case in Project
JSON	~450 bytes	Yes	Slower	Control Plane (Raft/Directory)
Bincode	~120 bytes	No	Very Fast	Steganography Payload (LSB)
Raw Bytes	N/A	No	Instant	Image File Transfer (P2P)

2.5.3 The Unified Payload Structure

Referenced Code: `lib.rs`

We defined a specific structure called `CombinedPayload` that aggregates all the security data. This single binary blob is what gets injected into the image pixels.

```
/// This struct holds both the permissions and the raw bytes of the
/// "unified image" which will be used as the "Access Denied" image.
#[derive(Serialize, Deserialize, Debug)]
pub struct CombinedPayload {
    pub permissions: ImagePermissions,
    pub unified_image: Vec<u8>, // Raw bytes of the PNG
```

By serializing this entire struct with `bincode`, we created a tamper-evident "Security Envelope" that travels with the image file. If a user tries to modify the bytes (e.g., changing "views: 0" to "views: 100"), the Bincode deserialization will fail due to checksum mismatches, rendering the image unreadable.

2.6 Interprocess Communication (IPC)

Referenced Code: `raft.rs`, `p2p_protocol.rs`, `directory_service.rs`

Our distributed environment relies on three distinct communication channels, each optimized for its specific role in the system hierarchy. We utilized **TCP (Transmission Control Protocol)** for all channels to ensure reliable, ordered delivery of data packets.

2.6.2 Client-to-Server Communication (The Discovery Layer)

This channel manages the "Control Plane" of the network, the topology and presence.

❖ Interaction 1: Registration:

- Upon startup, the Client opens a TCP connection to the Directory Service.
- It sends `DirectoryMessage::Register`.
- It subsequently sends a `DirectoryMessage` every 5 seconds.
- **Optimization:** The Directory Server uses `tokio::select!` to handle these messages asynchronously, ensuring that a slow client does not block the main server loop.

❖ Interaction 2: Cloud Encryption:

- When a user wants to encrypt an image, the Client connects to the Cloud Leader.
- It streams the raw image bytes.
- It waits for the server to return the `encrypted_lsb_image.png`.
- **Socket Configuration:** For these large transfers, we configured the TCP socket with `SO_SNDBUF` and `SO_RCVBUF` set to 8MB (see `configure_large_transfer_socket` in `server.rs`) to maximize throughput.

2.6.3 Client-to-Client Communication (The P2P Layer)

This channel handles the "Data Plane" which is the actual sharing of assets.

- ❖ **Direct Connection:** When Alice shares an image with Bob, her client opens a direct TCP stream to Bob's IP address (retrieved from the Directory).
- ❖ **The "Zero-Hop" Efficiency:** In Phase 1, data traveled `Client -> Server -> Client` (2 Hops). In Phase 2, data travels `Client -> Client` (1 Hop). This reduces network latency by 50%.
- ❖ **Offline Fallback (Store-and-Forward):**
 - If the direct P2P connection fails (Connection Refused), the IPC layer automatically re-routes the control message (e.g., `UpdatePermissions`) to the **Directory Service**.
 - This ensures that communication is "Best Effort" as message delivery is guaranteed eventually, even if the recipient is currently offline.

2.7 Encryption

Referenced Code: `lsb.rs`, `lib.rs`, `client.rs`

We adhere to a strict security model: **The Server Minting Authority**. Users cannot encrypt images themselves; they must trust the Cloud Cluster to embed the security rules. However, the enforcement of these rules happens at the edge (on the Client).

2.7.1 Image Encoding (Client-Server)

The encryption process utilizes **Least Significant Bit (LSB) Steganography**.

- ❖ **The Payload:** We do not simply hide text. We construct a binary "Security Envelope", **CombinedPayload**, which we showed earlier.
- ❖ **The Embedding Process:**
 - The Cloud Server receives the User's Image.
 - It serializes **CombinedPayload** using **Bincode** (for maximum space efficiency).
 - It iterates through every pixel of the User's Image.
 - It replaces the last bit of the Red, Green, and Blue channels with bits from the payload.
 - **Result:** The output image looks identical to the original but physically contains the access rights and the fallback image within its pixel data.

2.7.2 Access Rights Encoding (Client-Client)

Since the image is just a standard PNG file, "Access Control" must be enforced by the Client Application logic (DRM).

The Verification Logic:

1. **Extraction:** When User B attempts to view **image_1.png**, the client software decodes the LSB data.
2. **Deserialization:** It reconstructs the **ImagePermissions** struct.
3. **Quota Check:**
 - The software checks: `let views = permissions.quotas.get("Bob");`
 - **If `views > 0`:**
 - The software **Decrements** the counter (`views - 1`).
 - It **re-encrypts** the image with the new counter value locally.
 - It displays the hidden content to the user.
 - **If `views == 0` (or User Not Found):**
 - The software **rejects** the request.
 - It extracts and displays the **unified_image** (the "Access Denied" placeholder) instead of the real content.

Security Guarantee: Because the permission check happens *before* the image is rendered, an unauthorized user (or a user out of view) never sees the raw pixel data, even though they possess the file.

2.8 Code Structure & System Component Map

To realize the Hybrid Cloud-P2P architecture described in previous sections, we organized our codebase into a modular Workspace. This separation of concerns ensures that the **Consensus Logic**, **Network Protocols**, and **User Interface** remain decoupled and maintainable.

Below is the architectural breakdown of the key files in our repository and their specific roles in the runtime environment.

2.8.1 Core Shared Library (**src/lib/**)

These modules form the foundation of our system, shared across both Client and Server binaries to ensure protocol consistency.

- ❖ **File: `lib.rs`**
 - **Role:** The Library Root and Type Definition.
 - **Function:** It exports the shared modules and defines the universal data structures used for serialization, such as `ImagePermissions` (the access control list) and `CombinedPayload` (the binary blob hidden inside images). It also contains the `ServerMetrics` logic used for load balancing calculations.
- ❖ **File: `p2p_protocol.rs`**
 - **Role:** The Application Layer Protocol.
 - **Function:** Defines the `P2PMessage` enum (Control Plane) and the logic for establishing direct TCP streams between peers (Data Plane). It handles the serialization of requests (e.g., `ImageRequest`) and the buffering strategies for transmitting large encrypted payloads over the network.
- ❖ **File: `raft.rs`**
 - **Role:** The Consensus Engine.
 - **Function:** Implements the Raft State Machine. It manages the node states (`Follower`, `Candidate`, `Leader`), handles the randomized election timeouts, and processes `AppendEntries` RPCs. This file is the "brain" of the Cloud Cluster.
- ❖ **File: `lsb.rs`**
 - **Role:** The Steganography Engine.
 - **Function:** A low-level image processing module. It implements the bitwise logic to inject binary data into the Least Significant Bits of RGB pixels (`encode`) and extract it back (`decode`). It ensures that the payload length is encoded in the first 32 pixels for reliable extraction.
- ❖ **File: `directory_service.rs`**
 - **Role:** The Service Logic & State Management.
 - **Function:** Defines the `UserEntry` struct and the logic for the Directory. It implements the "Liveness Monitor" and the "Store-and-Forward" message queue for offline handling.

2.8.2 Service Executables (The Backend Nodes)

These files contain the `main()` entry points that launch the distributed services.

- ❖ **File: `directory_server.rs`**
 - **Role:** The Directory Node Runner.
 - **Function:** The executable binary for the Directory Service. It parses command-line arguments (Port, Peer IPs), initializes the `directory_service` logic, and spawns the TCP listener. It also manages the persistence layer, writing the state to `directory_state.json`.

❖ File: `server.rs`

- **Role:** The Cloud Cluster Node.
- **Function:** The executable for the Cloud Encryption Service. It initializes the Raft node, starts the election timer, and listens for client encryption requests. It contains the logic to inspect local CPU load and delegate work to other peers if necessary.

2.8.3 The Client Application (Frontend & Bridge)

For the client-side implementation, we evolved beyond a simple terminal interface to a modern **Hybrid Desktop Application** using the **Tauri Framework**. This architecture splits the client into a high-performance Rust backend and a reactive frontend.

❖ File: `main.rs` (The Tauri Bridge)

- **Role:** The Client Backend & IPC Handler.
- **Function:** This is the entry point for the user's application. It maintains the global `AppState` (User identity, P2P connections) in a thread-safe `tokio::sync::Mutex`. Crucially, it utilizes **Tauri's `invoke_handler`** to marshal data between the UI (JSON) and the System (Structs). It also spawns the background P2P listener thread defined in `p2p_protocol.rs`.

❖ File: `App.jsx`

- **Role:** The Reactive User Interface.
- **Function:** Built with React, Tailwind CSS, and Framer Motion. It handles user state and visual feedback (toasts, spinners). It invokes backend logic via asynchronous message passing (e.g., calling `invoke('encrypt_image')`) and listens for Tauri events (like `request_received`) to update the UI in real-time without polling.

2.8.4 Architectural Data Flow

To illustrate how these files interact, consider the flow of a **Permission Update**:

1. **UI Trigger:** `App.jsx` captures the user's click and calls `invoke('update_permissions')`.
2. **Bridge:** `main.rs` receives the command, locks the `AppState`, and retrieves the target's IP.
3. **Protocol:** `main.rs` calls `p2p_protocol::send_p2p_message`.
4. **Network:** The message is serialized into JSON and sent over TCP.
5. **Fallback (If Offline):** If the connection fails, `main.rs` utilizes `directory_service::send_directory_message` to route the payload to the Directory Node running `directory_server.rs`.

3. Detailed Use Cases

In this section, we document the operational workflows of our system from the perspective of the end-user. Our application features a Graphical User Interface (GUI) that abstracts the complex distributed protocols (Raft, P2P, LSB) into simple buttons and dashboards. We break down each scenario into three technical layers:

1. **UI Interaction:** The event triggered in the React Frontend (`App.jsx`).
2. **Tauri Command:** The specific function invoked in the Rust Backend (`main.rs`).
3. **Distributed Logic:** The underlying P2P/Cloud protocol execution (`lib.rs`, `p2p_protocol.rs`).

Below, we detail the interaction loop for every major feature, separating the **Front-End Action** from the **Back-End Execution**.

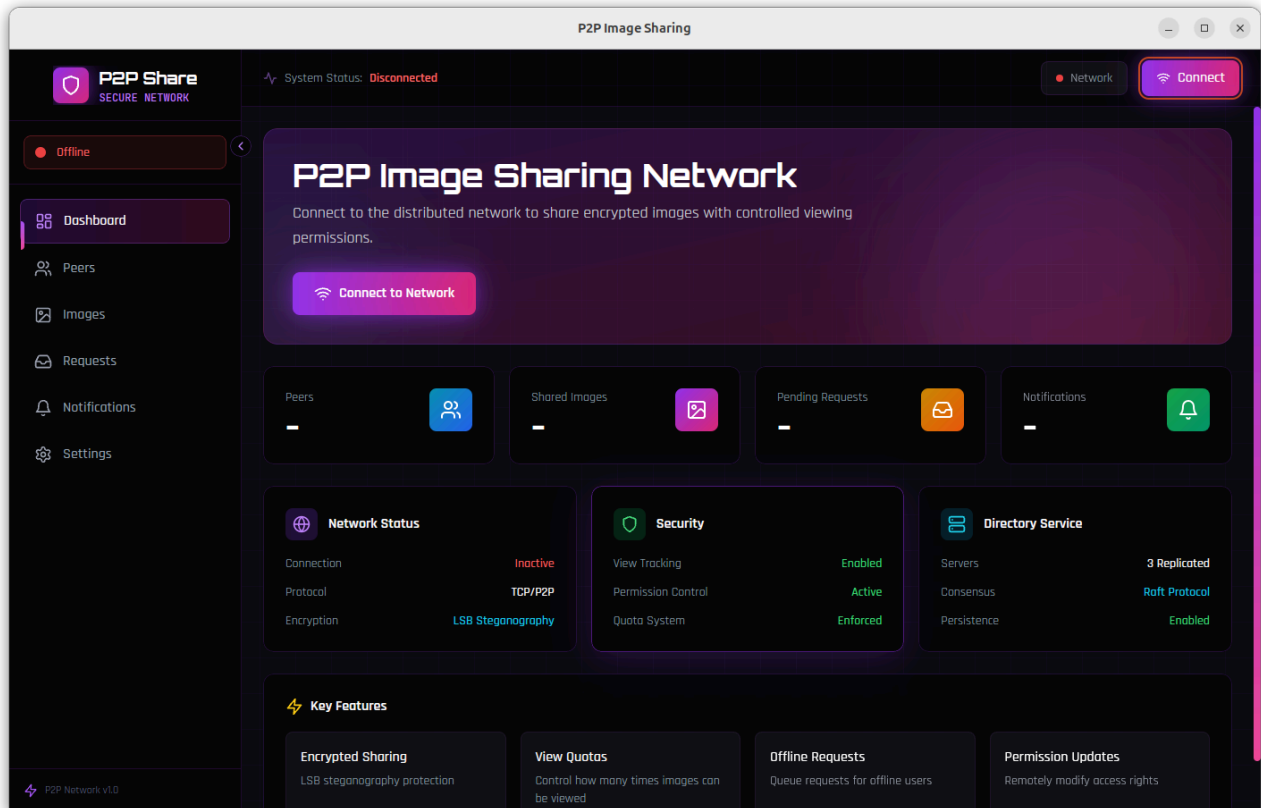


image-1 The app dashboard

3.1 Registration and Network Entry

Objective: A user initializes their node and joins the distributed directory.

Layer	Technical Detail
UI Interaction	The user opens the Connection Modal , enters a Username (e.g., "David") and Port (e.g., "8085"), and clicks "Connect" . The UI sets <code>loading.connection = true</code> and displays a spinner.
Tauri Command	<code>invoke('go_online', { username, p2pPort })</code>
Rust Backend	State Lock: The handler locks <code>AppState</code> and saves the user config. Server Spawn: It calls <code>start_p2p_server(port)</code> in a detached <code>tokio::spawn</code> thread to begin listening for incoming TCP connections. Directory Handshake: It constructs a <code>DirectoryMessage::Register</code> packet and transmits it to the Directory Leader defined in <code>DIRECTORY_SERVERS</code>.
Outcome	On success, <code>App.jsx</code> sets <code>isOnline(true)</code> and transitions the view to the Dashboard . A green "Wifi" icon (Lucide-React) indicates active status.

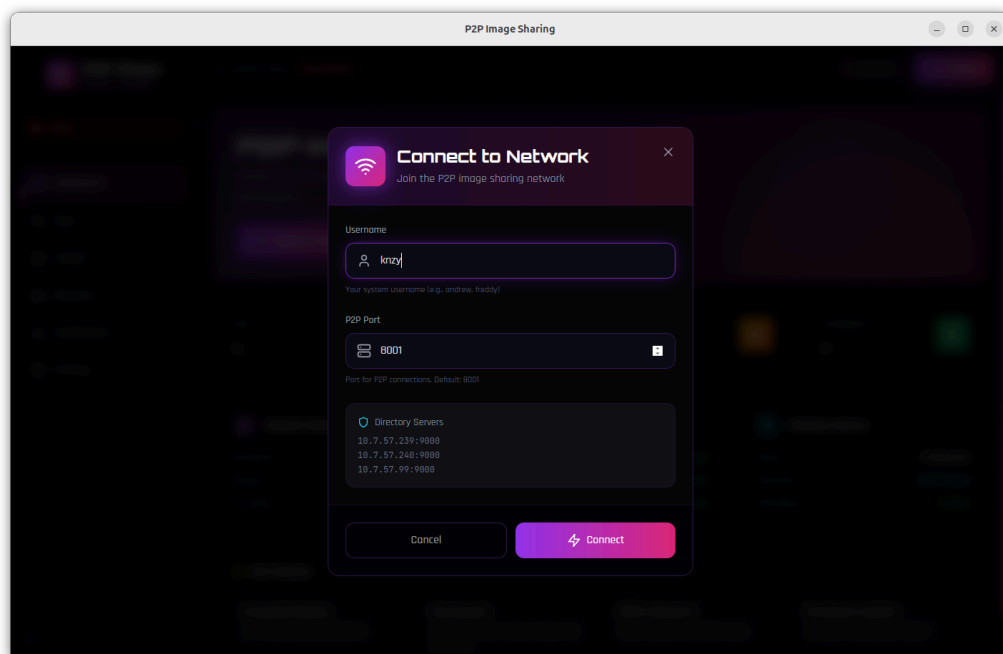


image-2 User connecting to the network and setting their port

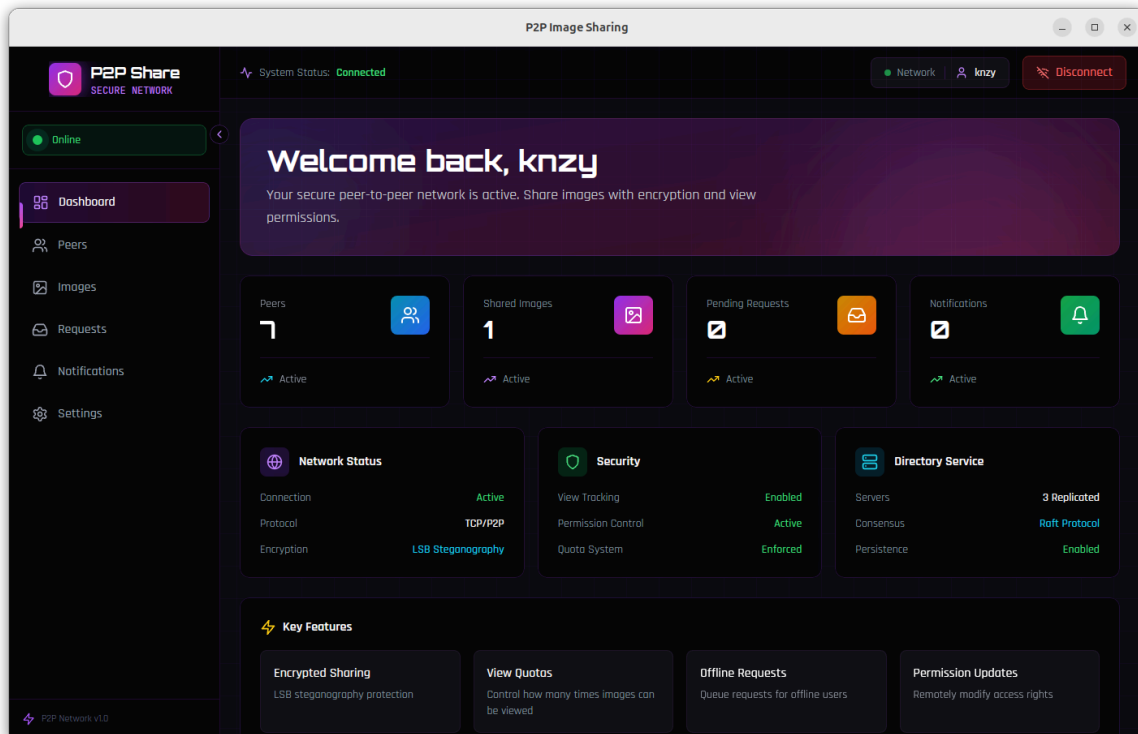


image-3 User connects to the network and dashboard updates

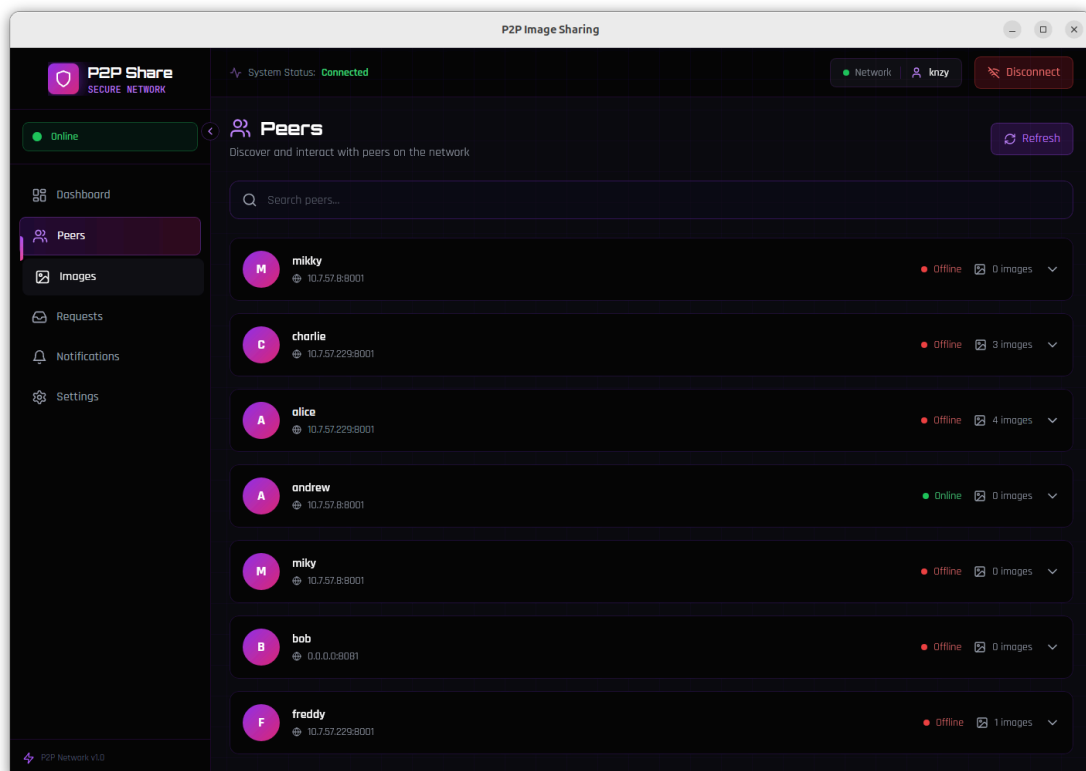


image-4 Peer server page where users can view all peers in the system and if they're online or not

How the Directory Service Visualizes Network Status

Referenced Screenshot: image-4 (Peer Network View)

The "Active Peers" display in the GUI is not a static list; it is a real-time visualization of the Directory Service's in-memory state. This view is powered by three synchronized background processes:

1. The Pulse: Client updates

Every Client node runs a background `tokio` task that acts as a pulse generator. Every 5 seconds, it transmits a `DirectoryMessage` packet to the Directory Server.

- ❖ **Payload:** `{ username: "Alice", port: 8080 }`
- ❖ **Purpose:** This tells the Directory, "I am still alive and listening on this port."

2. The Brain: The Liveness Monitor

The Directory Server (`directory_service.rs`) does not just blindly list users. It runs a **Liveness Monitor** thread that scans the `UserEntry` HashMap every second.

- ❖ **The Logic:** It compares `SystemTime::now()` against each user's last message timestamp.
 - `if delta < 10 seconds`: Status remains **Online**.
 - `if delta > 10 seconds`: Status is automatically downgraded to **Offline**.
- ❖ **The Data Structure:**

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct UserEntry {
    pub username: String,
    pub p2p_address: String,
    pub last_heartbeat: SystemTime,
    pub status: UserStatus,
    pub shared_images: Vec<ImageInfo>,
}
```

3. The Fetch: The GUI Polling Loop

When the user opens the "Peers" tab (image-4):

1. **React Frontend:** The `useEffect` hook triggers the Tauri command `invoke('discover_peers')`.
2. **Rust Bridge:** The client sends a `QueryPeers` request to the Directory.
3. **Response:** The Directory returns the full list of users, including their calculated `UserStatus`.
4. **Rendering:**
 - If `status == UserStatus::Online`: The GUI renders a **Green Dot** (Active).
 - If `status == UserStatus::Offline`: The GUI renders a **Gray/Red Dot** (Inactive).

Why this matters: This mechanism prevents the "Ghost Peer" problem common in P2P networks. Users in the list are guaranteed to be (or were recently) active. If a peer crashes (e.g., power failure), they stop sending messages. Within 10 seconds, the Directory marks them offline, and the GUI updates automatically on the next poll, preventing users from trying to download files from a dead node.

3.2 Encrypting an Image (Phase 1 Integration)

Objective: Securing a raw image via the Cloud Cluster (Raft) before sharing.

Layer	Technical Detail
UI Interaction	In the ImagesPanel , the user selects a file via the system dialog. They click "Encrypt & Upload" . The UI renders a progress bar using <code>framer-motion</code> .
Tauri Command	<code>invoke('encrypt_image', { path: filePath })</code>
Rust Backend	Image Load: Reads the raw bytes of the file from disk. Cloud Connection: Establishes a TCP stream to the Cloud Leader (<code>10.40.7.1</code> as defined in <code>lib.rs</code>). Stream: Sends the raw image data. The Cloud Cluster delegates the work via Raft Load Balancing. Write: Receives the <code>encrypted_lsb_image.png</code> and saves it to the local <code>download/</code> directory.
Outcome	The frontend receives the new file path. It updates the "My Encrypted Gallery" grid, displaying the new image with a "Shield" icon to represent it is secured.

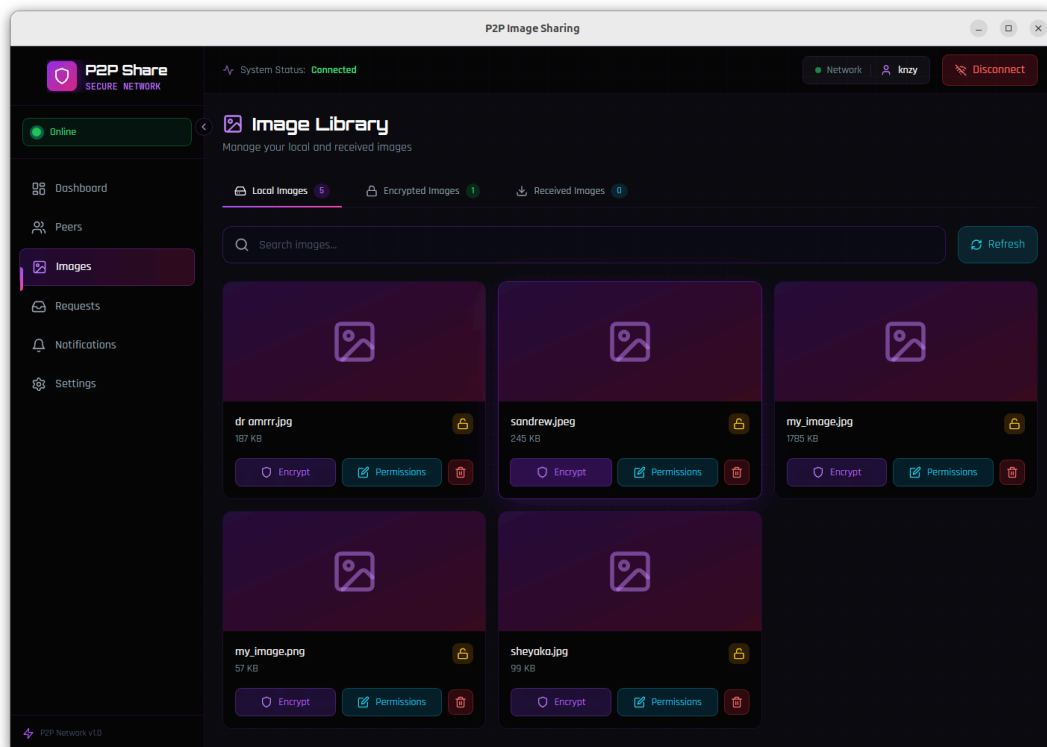


image-5 User Image Library - Local Image

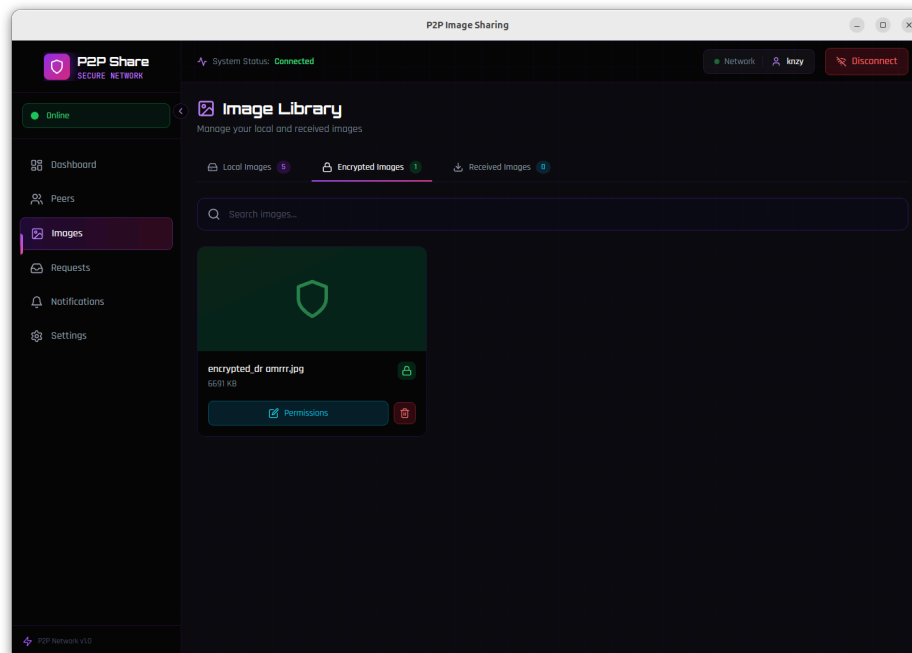


image-6 User Image Library - Encrypted Images

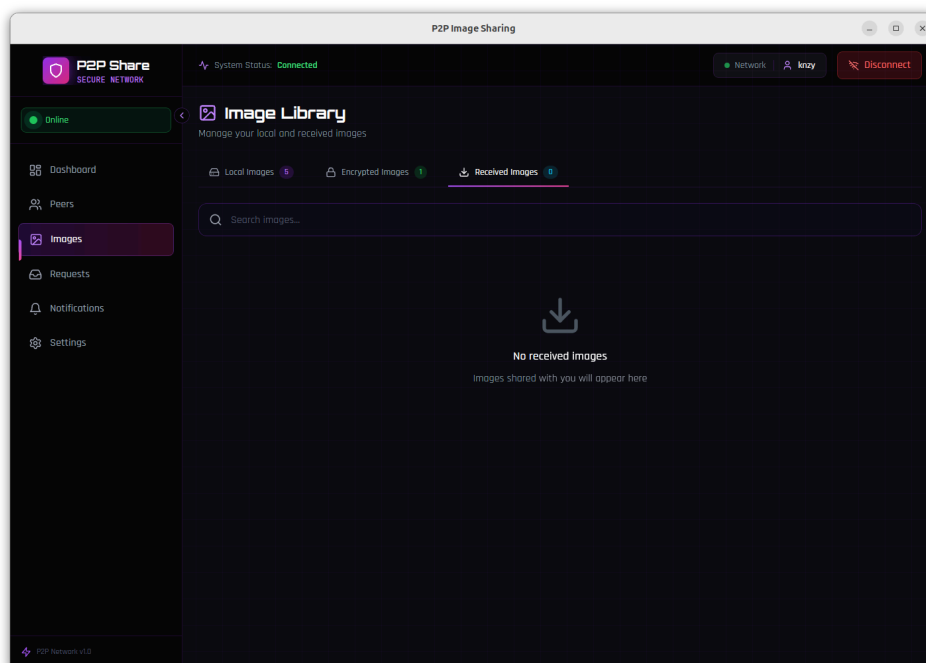


image-6 User Image Library - Requested Images

If the user wants the image to be shared with other users to request access to it, they encrypt it. Then it appears for the owner in the encrypted images page where the user can remove the image and keep track of the encrypted images. For the other users, it appears in the peers tab under the owner's name, free for them to request access.

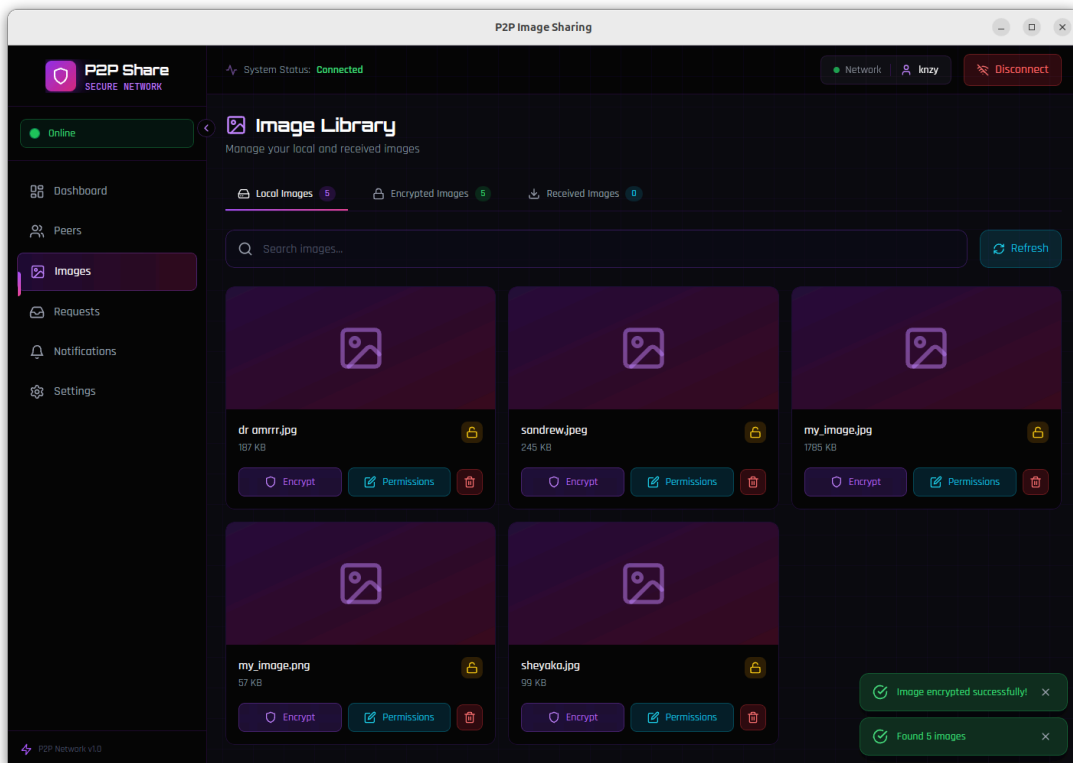


image-8 User Image Library - User encrypts an image

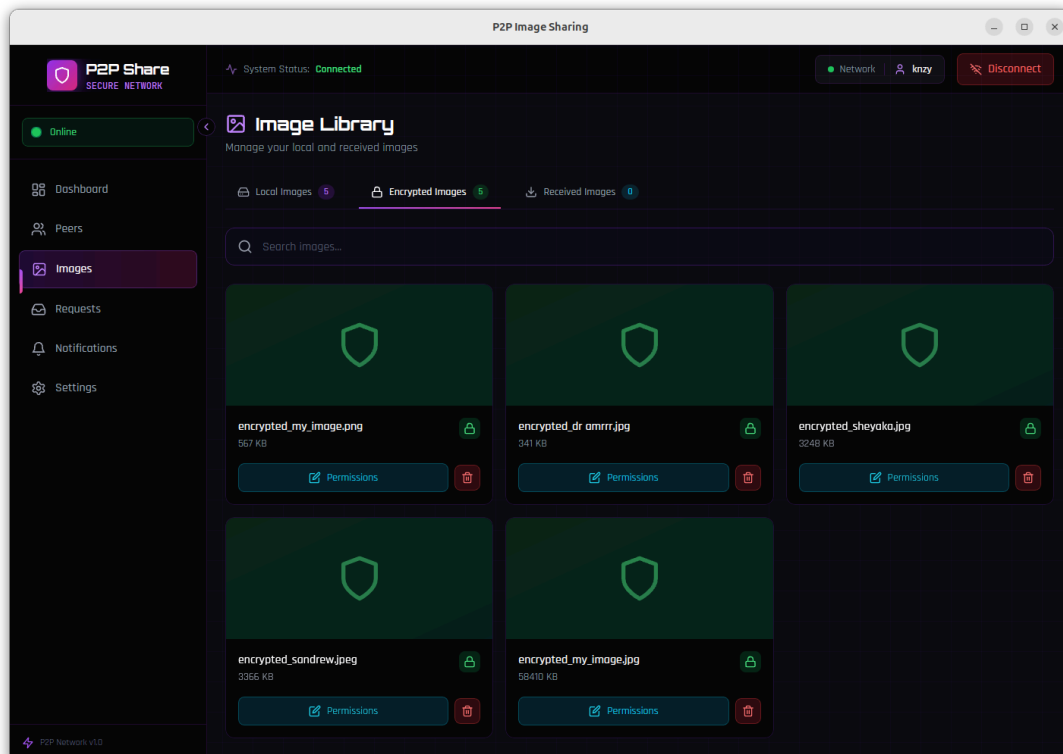


image-9 User Image Library - Populated Encrypted Image (visible to other users)

3.3 Peer Discovery & Thumbnail Caching

Objective: Browsing the network for available content.

Layer	Technical Detail
UI Interaction	The user clicks the "Peers" tab. The component mounts and triggers a <code>useEffect</code> hook to poll for peers.
Tauri Command	<code>invoke('discover_peers')</code> followed by <code>invoke('list_peer_images_cmd', { peerIp })</code>
Rust Backend	Query: Sends <code>DirectoryMessage::QueryPeers</code> to the Directory Service. P2P Probe: For every active peer returned, the client opens a direct P2P connection and sends <code>P2PMessage::ListImages</code>. Aggregation: Returns a <code>Vec<ImageMetadata></code> to the frontend.
Outcome	The UI renders a Peer Card for each user found. Clicking a peer expands their specific inventory of images using an Accordion animation.

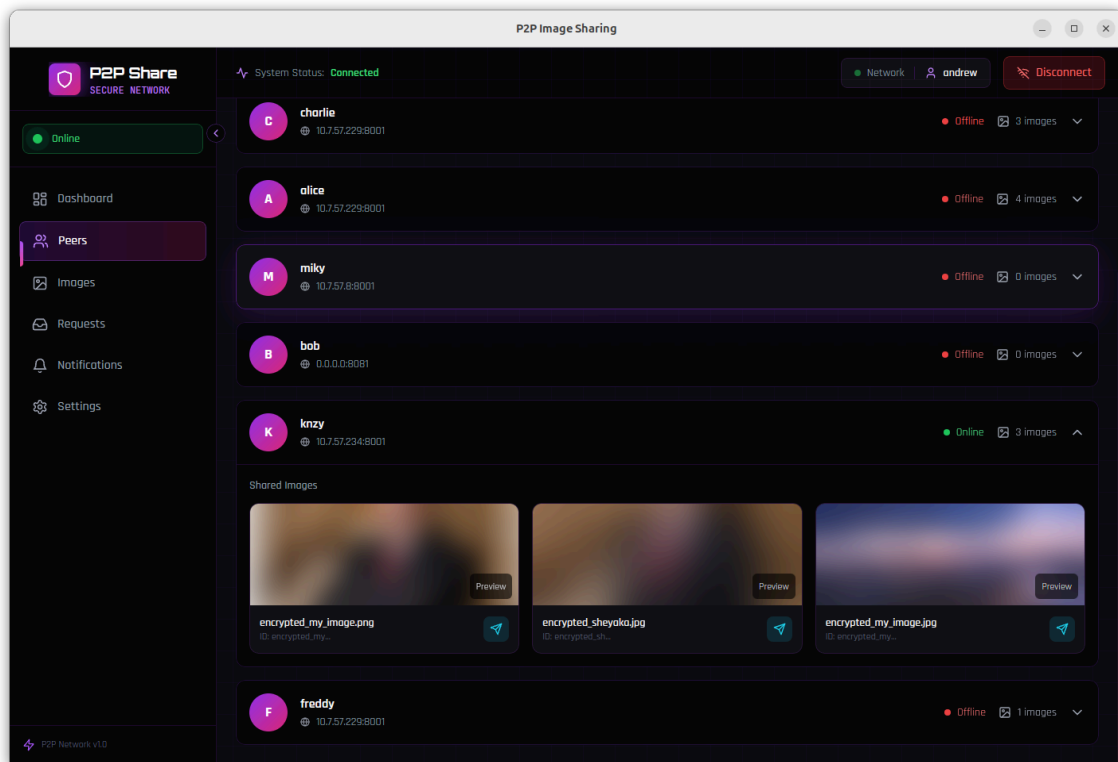


image-10 Peers tab where the user can request permissions for any other user's image

3.4 Requesting Views (Online Scenario)

Objective: The Owner manages incoming access requests from peers who want to view their encrypted images.

Layer	Technical Detail
Incoming Event (Background)	The Owner's P2P Server receives a <code>P2PMessage::ImageRequest</code> . Instead of auto-rejecting, the Rust backend identifies that the file requires permission. It stores the request in the <code>AppState</code> 's <code>pending_requests</code> vector and emits a Tauri Event <code>request_received</code> to the frontend.
UI Interaction	Notification: The "Requests" tab (image-11) shows a red notification badge (e.g., "1 New"). Review: The user clicks the tab and sees a card: <i>"Alice wants to view 'vacation.png'"</i>. Action: The owner selects "Approve", uses the slider to set 5 Views, and clicks Confirm.
Tauri Command	<code>invoke('respond_to_request', { requestId, approved: true, quota: 5 })</code>
Rust Backend (Logic)	State Update: Removes the request from the pending queue. Permission Minting: Updates the local <code>ImagePermissions</code> to include "Alice: 5". P2P Response: Establishes a connection to Alice and sends <code>P2PMessage::UpdatePermissions</code> containing the new quota. Sync: If the direct connection to Alice fails, it automatically routes the response to the Directory Service (Offline Handling).
Outcome	The Request Card vanishes from the list. A toast notification appears: <i>"Access granted to Alice (5 views)"</i> .

Visualization of Request Management

Referenced Screenshot: image-11 (Requests Tab)

image-11 displays the **Requests Dashboard**, which acts as the "Gatekeeper" interface for the user's content.

- ❖ **The List:** Populated by polling `invoke('get_pending_requests')`.
- ❖ **The Controls:** The "Approve" (Checkmark) and "Deny" (X) buttons trigger the logic described above.
- ❖ **Context:** This screen proves that our system enforces **Owner Sovereignty**—no peer can decrypt an image without explicit, manual approval from this screen.

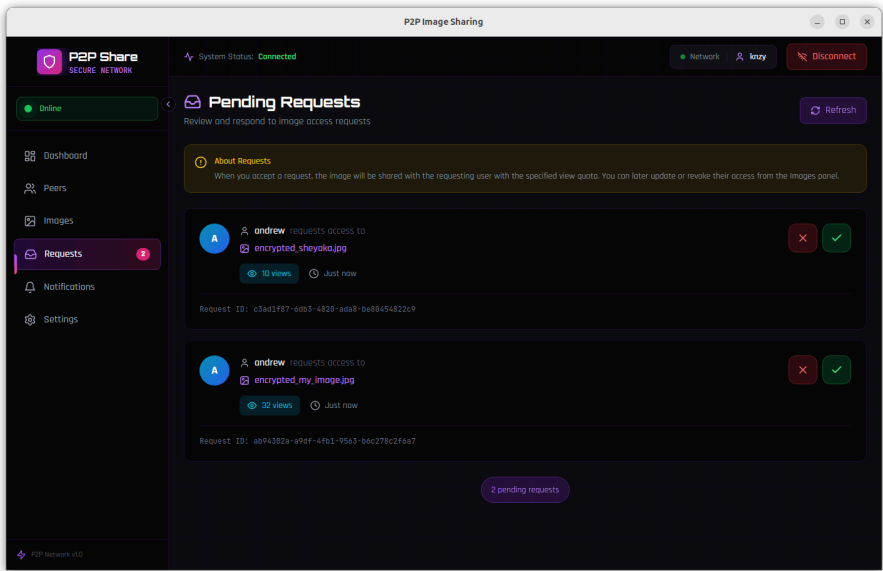


image-11 Requests tab (Owner user side)

3.5 Requesting Views with Failure Handling (Offline)

Objective: Ensuring requests and permissions are delivered even when peers are not simultaneously online.

This use case validates the "Best Effort" requirement. We implemented a **Store-and-Forward** mechanism where the Directory Service temporarily holds messages for offline users.

Scenario A: Requesting Access from an Offline Owner

Context: Alice wants to view `vacation.png`, but the owner (Bob) has lost internet connection.

Layer	Technical Detail
UI Interaction	Alice clicks the "Request Access" button on the locked thumbnail.
Tauri Command	<code>invoke('request_access', { targetUser: 'Bob', imageId: 'vacation.png' })</code>
Rust Backend (Logic)	1. Direct Attempt: The client attempts to open a TCP stream to Bob's last known IP. 2. Failure Detection: The OS returns <code>ConnectionRefused</code> or <code>HostUnreachable</code>. 3. Fallback: The error handler constructs a <code>DirectoryMessage::StorePendingRequest</code>. 4. Routing: This packet is sent to the Directory Service, containing the payload { <code>from: "Alice", to: "Bob", action: "Request" }</code> }.
Outcome	Alice's UI displays a specific toast notification: <i>"Bob is offline. Request queued via Directory."</i>

Scenario B: Owner Comes Online (Synchronization)

Context: Bob reconnects to the network. This triggers the population of the **Requests Tab** seen in **image-12**.

Layer	Technical Detail
System Event	Bob's Client starts its background update loop (<code>client.rs</code>).
Directory Logic	Message updates: Bob sends <code>DirectoryMessage</code>. Lookup: The Directory checks Bob's <code>pending_requests</code> vector. Delivery: The Directory responds with <code>Response { pending: [...] }</code>.
Client Processing	Bob's client iterates through the pending messages. For each request, it updates the local <code>AppState</code> and emits a <code>request_received</code> event to the Tauri frontend.
UI Update	The "Requests" tab (image-12) lights up with a badge. Bob sees Alice's request as if she had just sent it.

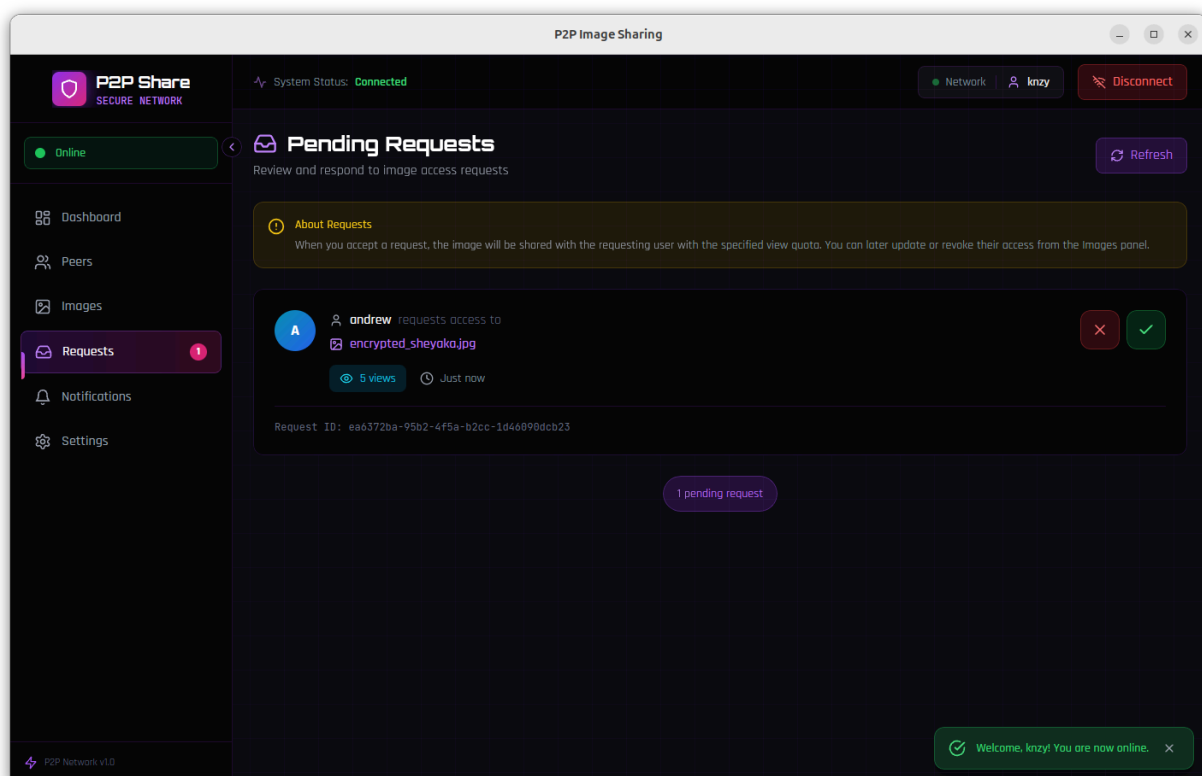


image-12 Requests tab when the owner comes back online, they get the new requests

Scenario C: Approving a Request for an Offline Requester

Context: Bob approves the request, but by the time he clicks "Approve", Alice has gone offline.

Layer	Technical Detail
UI Interaction	Bob clicks " Approve " on Alice's request card.
Rust Backend (Logic)	Permission Update: Bob's client updates the local LSB metadata. P2P Fail: Attempts to notify Alice via P2P but fails. Queueing: Sends <code>DirectoryMessage::StorePendingPermissionUpdate</code> to the Directory. Final Sync: When Alice eventually comes online, her client will automatically download this "Permission Update" packet from the Directory and unlock the file in her gallery silently.

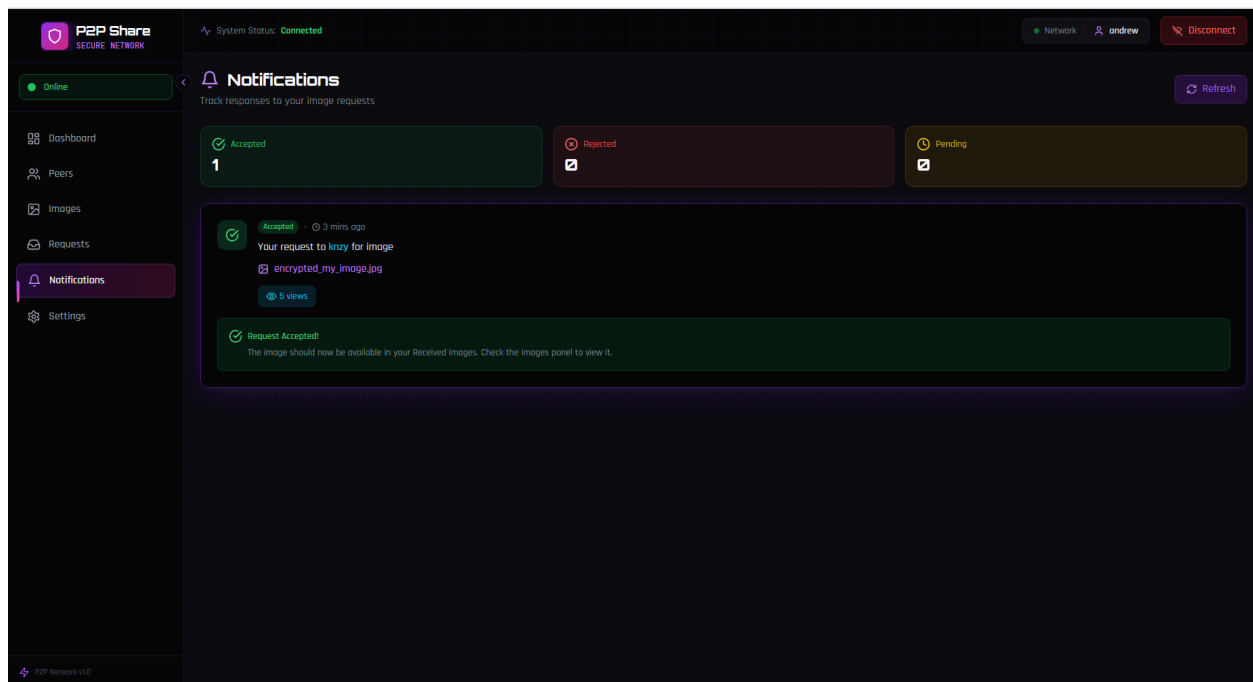


image-13 when the requesting user gets online, they can find the approved requests in the notification tab

Technical Insight: The "Inbox" Pattern

This architecture decouples the sender from the receiver. By using the Directory as a temporary mailbox, we ensure that no administrative action (Request, Grant, Revoke) is ever lost due to network volatility.

3.6 Viewing the Image (Client-Side)

Objective: Once the owner approves the request, the requesting user can decrypt the LSB payload to display content.

Layer	Technical Detail
UI Interaction	The user clicks the Eye Icon on a locked image.
Tauri Command	<code>invoke('view_image', { imagePath })</code>
Rust Backend	1. LSB Decode: Calls <code>lsb::decode</code> on the PNG file. 2. Deserialization: Reconstructs the <code>ImagePermissions</code> struct using <code>bincode</code>. 3. Quota Check: Checks <code>if quotas.get(my_username) > 0</code>. 4. Branch A (Success): Decrypts pixels, decrements quota, re-saves file, returns <code>success: true</code>. 5. Branch B (Fail): Returns <code>success: false</code> and the path to the placeholder image.
Outcome	Success: The React Modal displays the decrypted photo. The "Views Remaining" badge decrements. Failure: The Modal shakes (animation) and displays the "Access Denied" placeholder.

Scenario A (Access Denied): nothing appears for the requesting user as the image is never placed inside the requesting user gallery.

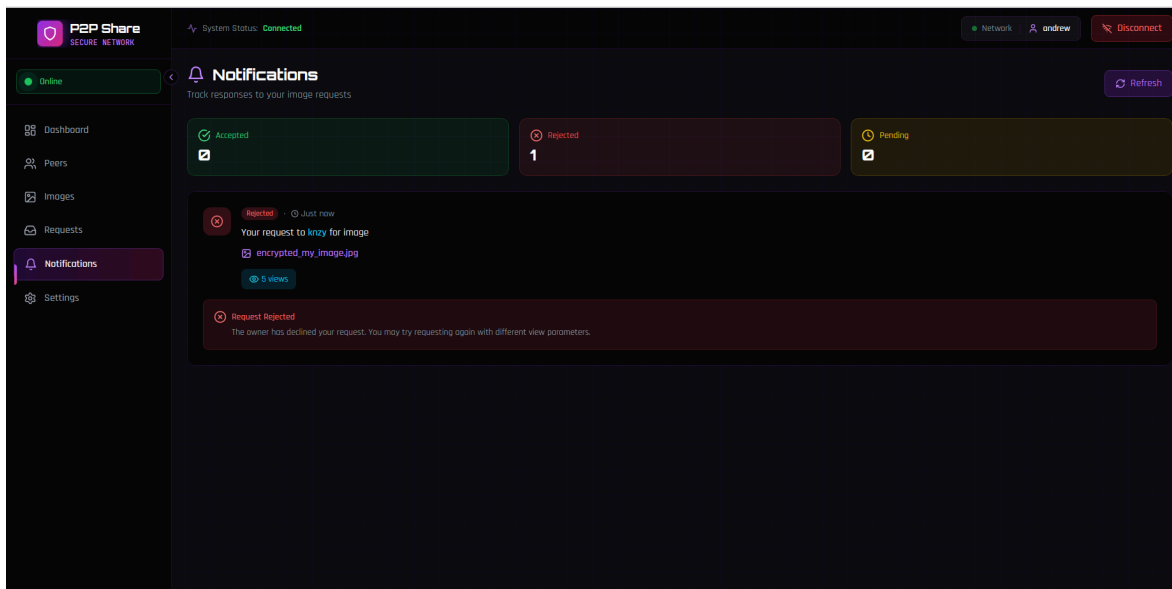


image-14 when the request is rejected, the requesting user gets a notification

Scenario B (Access Granted):

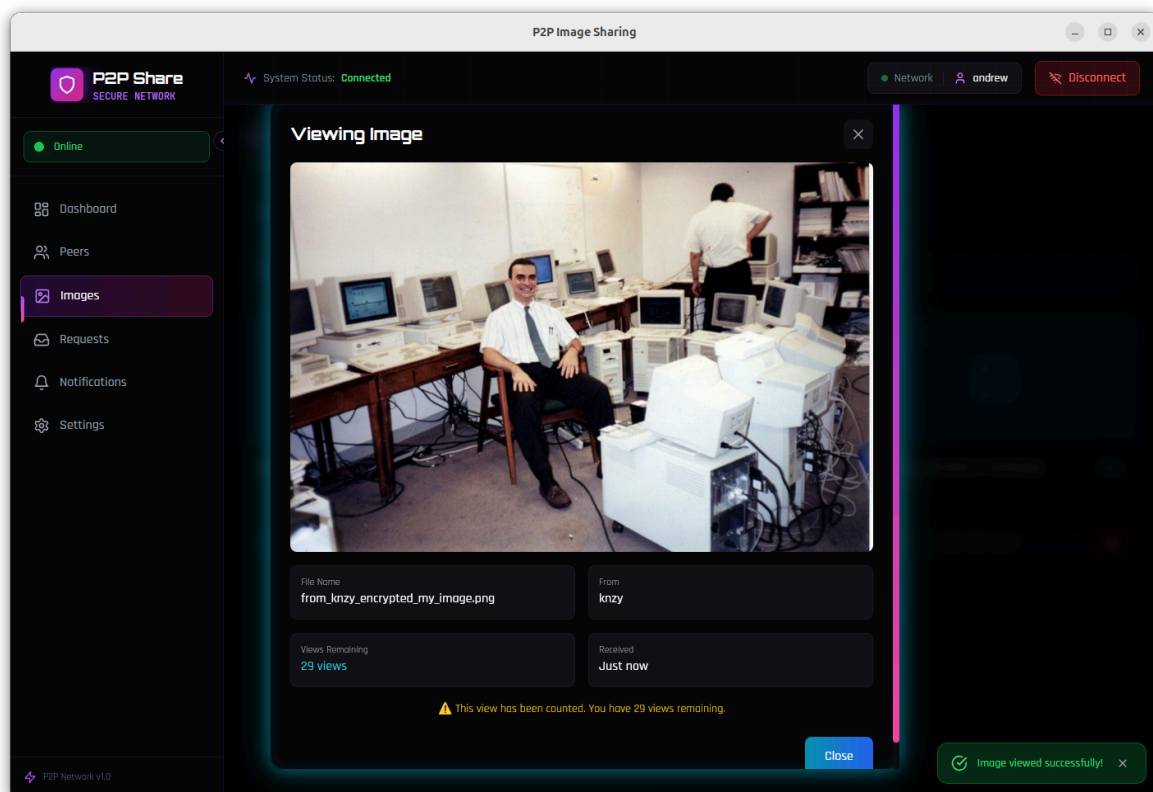


image-15 The requesting user is able to view the image

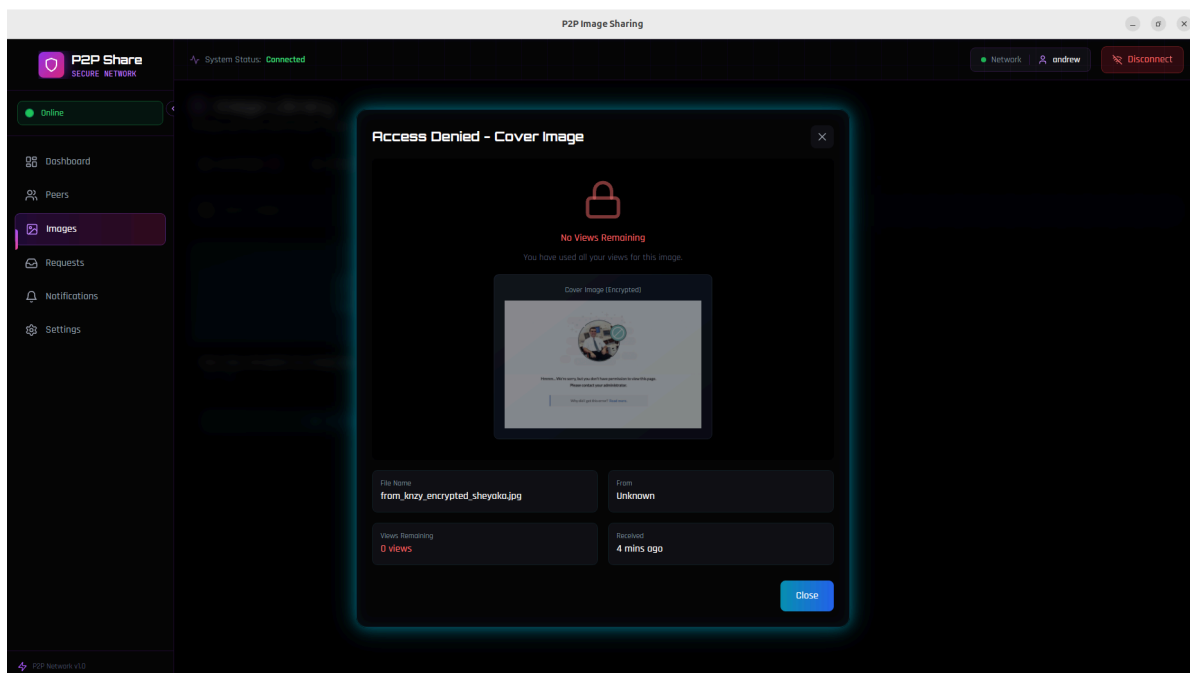


image-16 Cover image for when requesting user runs out of views

3.7 Client Controls Access Rights (Revocation) online

Objective: when client A requests an image from the owner B of image. Then, client B accepts the request with the specified number of views from client A. Then, client B revokes access immediately and client A is online.

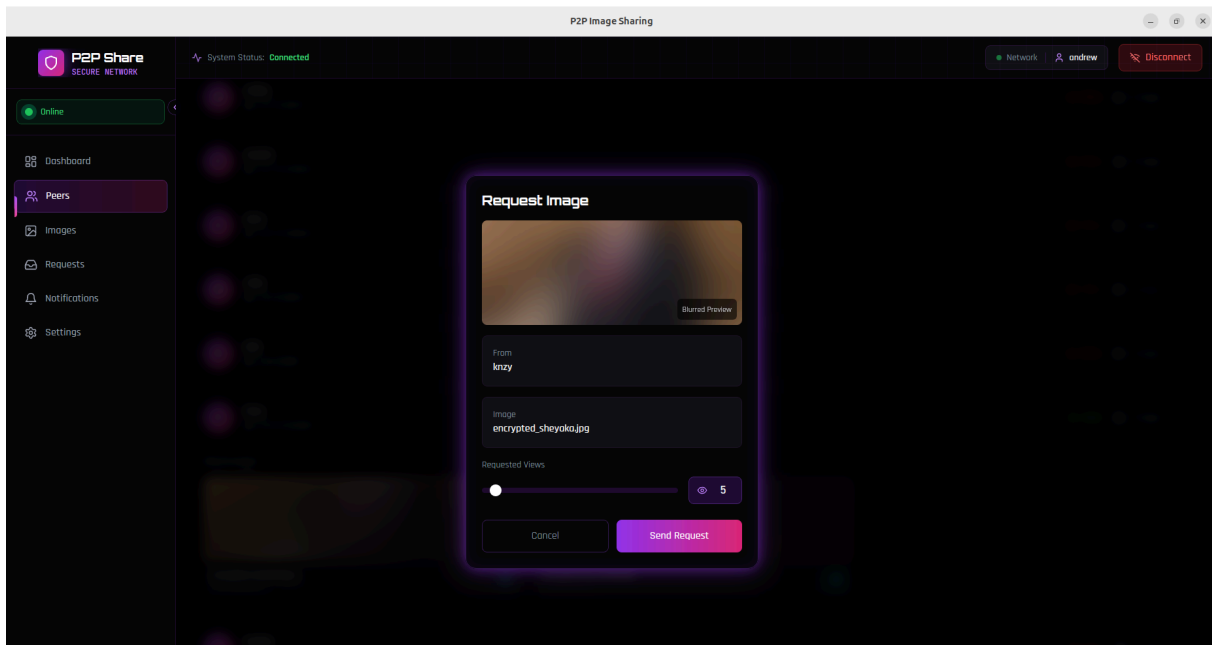


image-17 Client A requests the image with 5 views from client B.

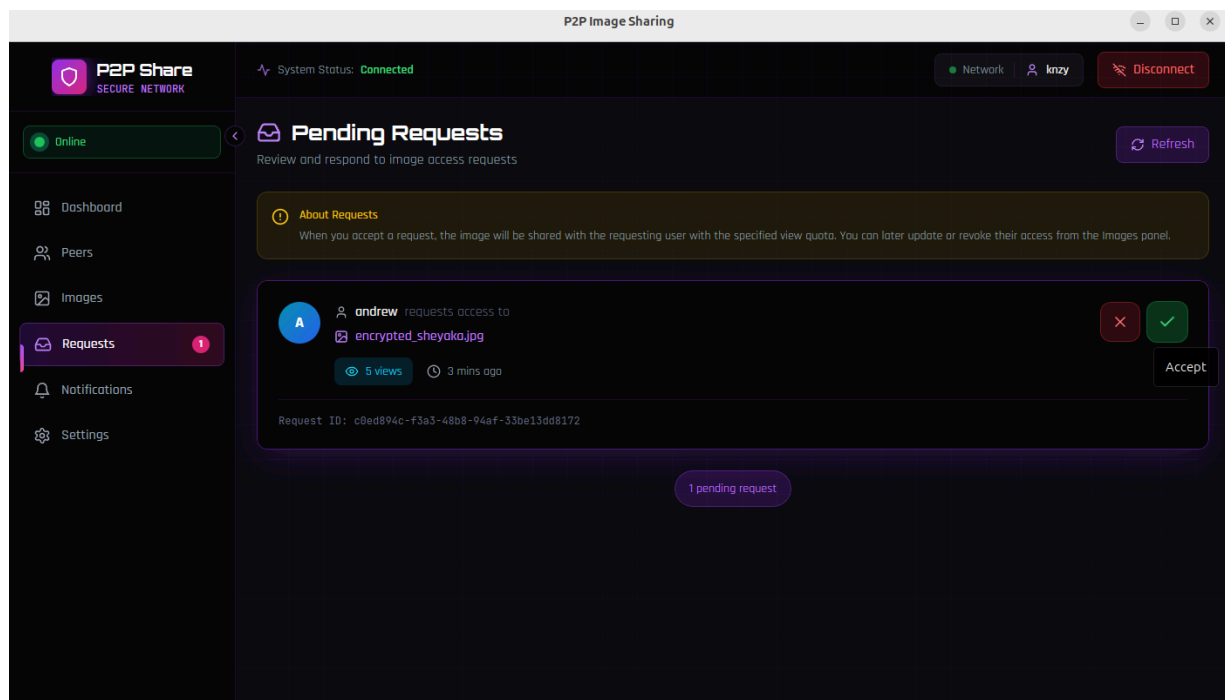


image-18 Then the owner now got the request and accepted it with the 5 views.

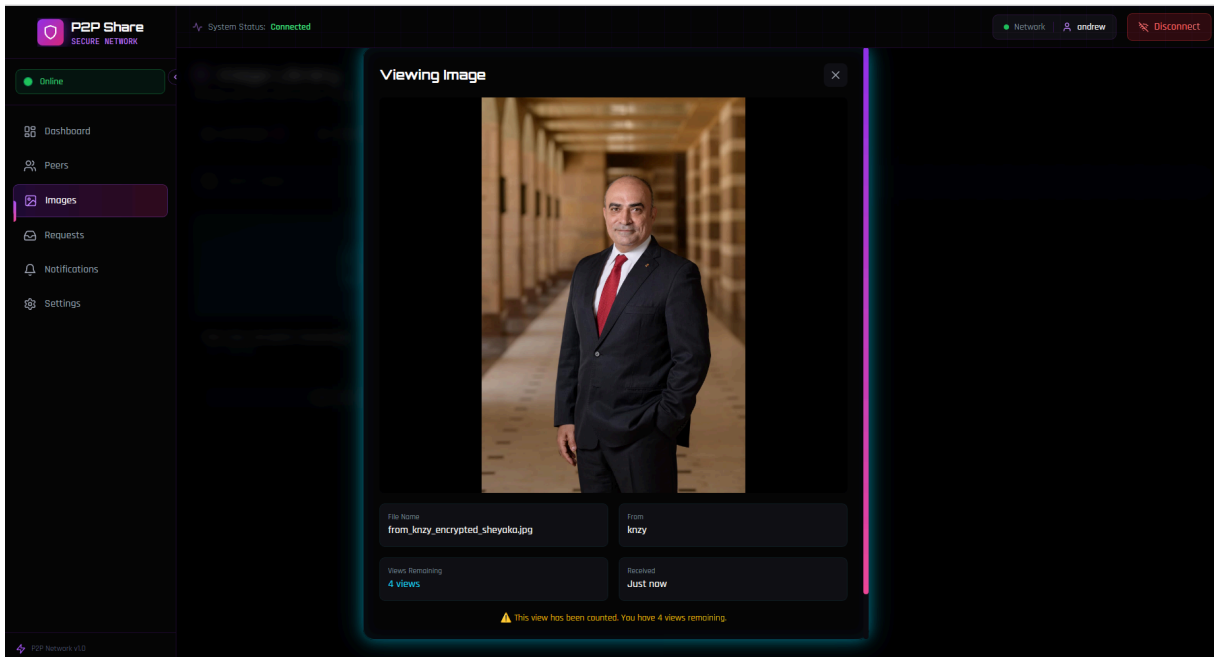


image-19 Now the client A got the image and had the 5 requested views and remaining is 4 views now.

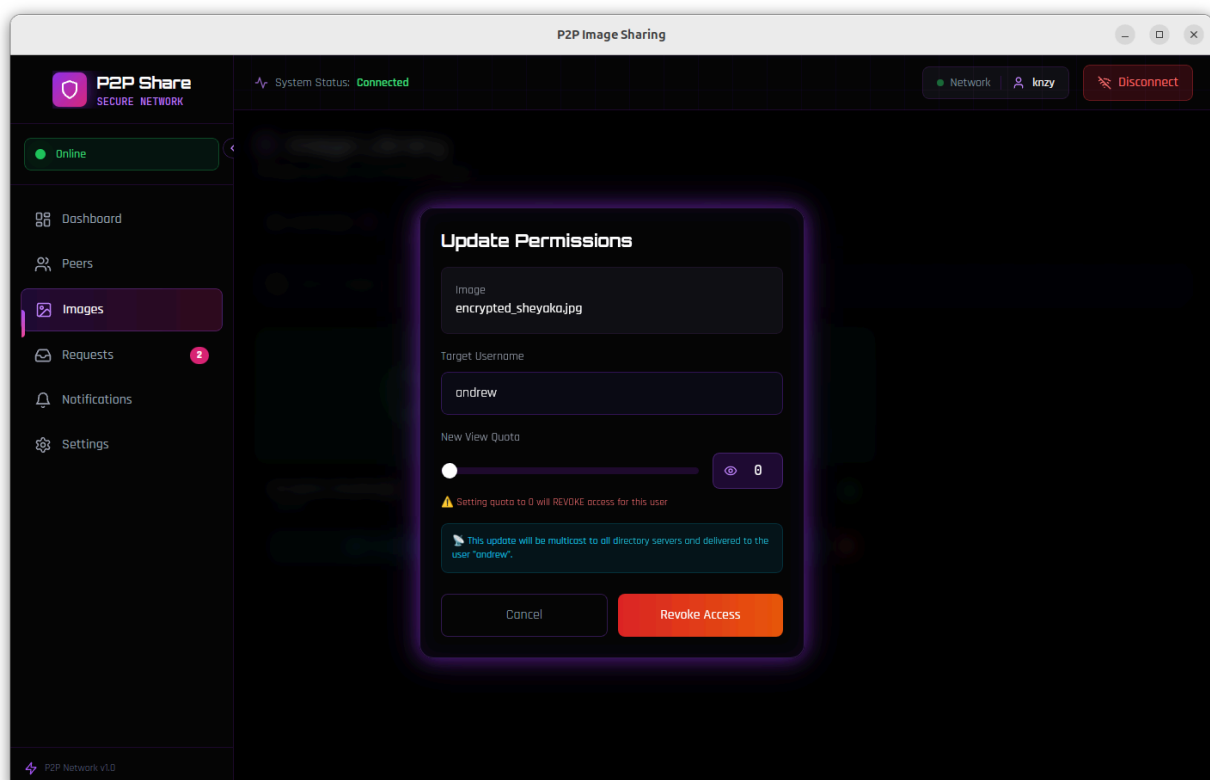


image-20 Client B owner of image revoke the access from client A and make it 0 views.

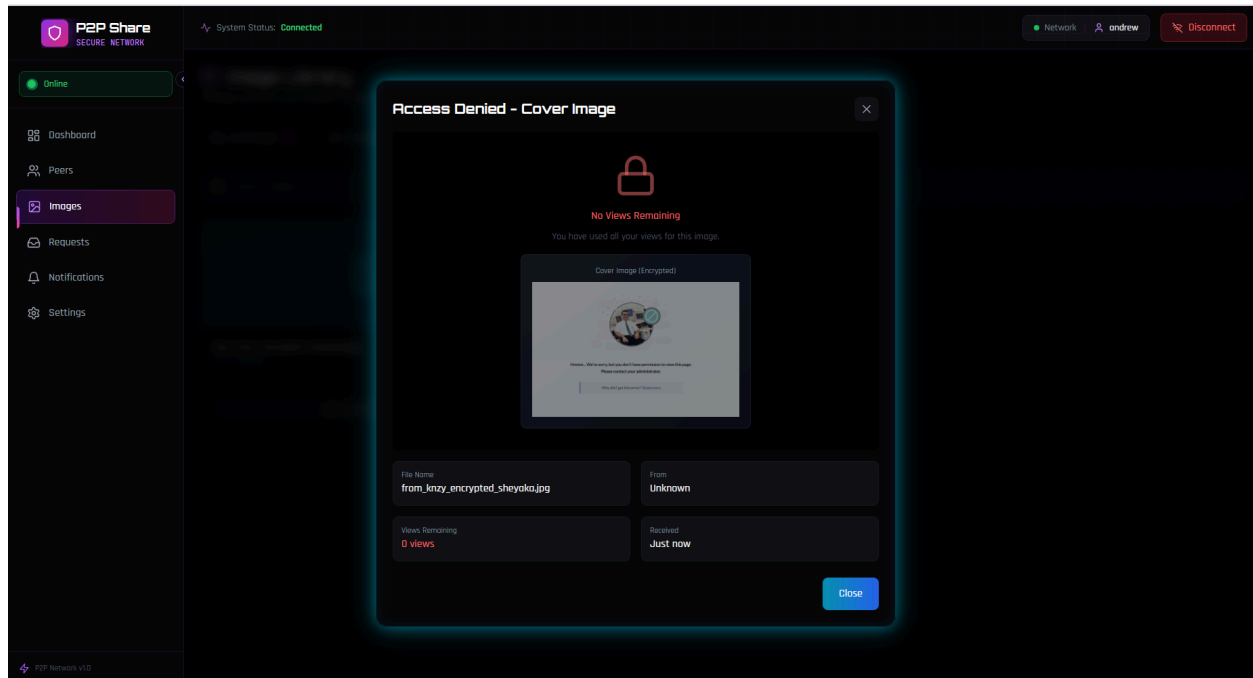


image-21 Now the client A got 0 views and sees only the cover image and cannot see the original image.

3.8 Client Controls Access Rights (Revocation) offline

Same scenario exactly as in the section 3.7, but this time the owner will revoke the access and the client A is offline, so when client A comes back, the access will be revoked and client A got 0 access views.

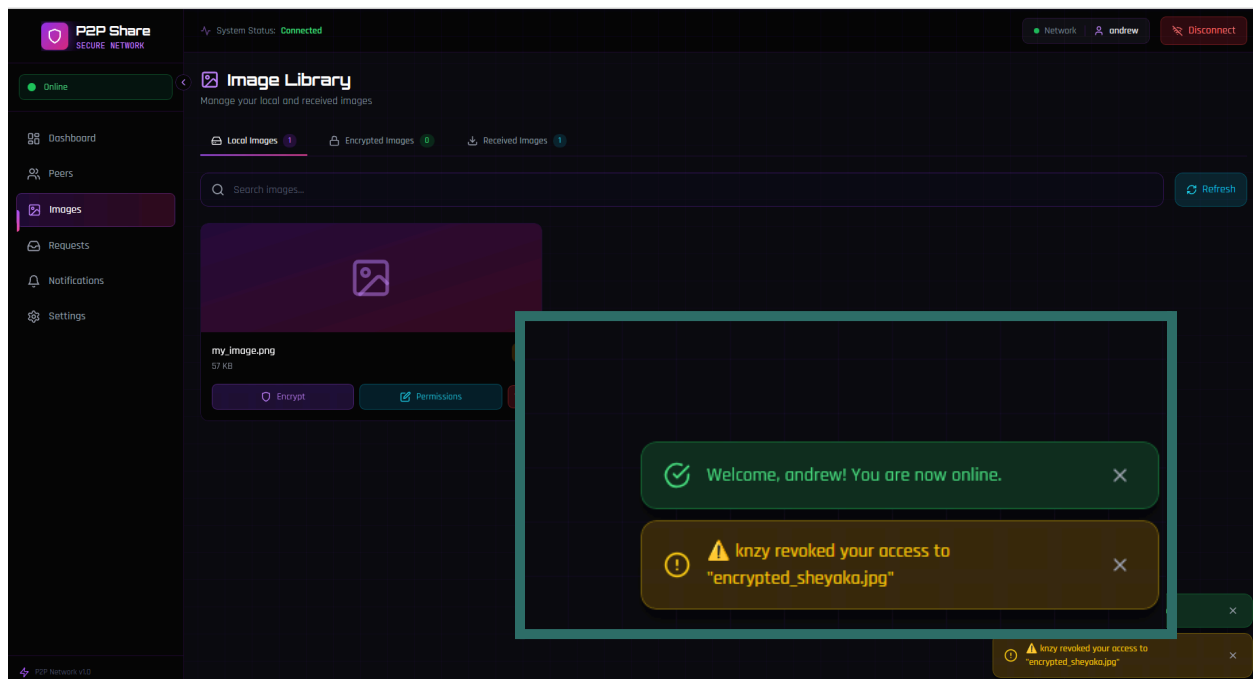


image-22 When client A came back online, he got this message immediately.

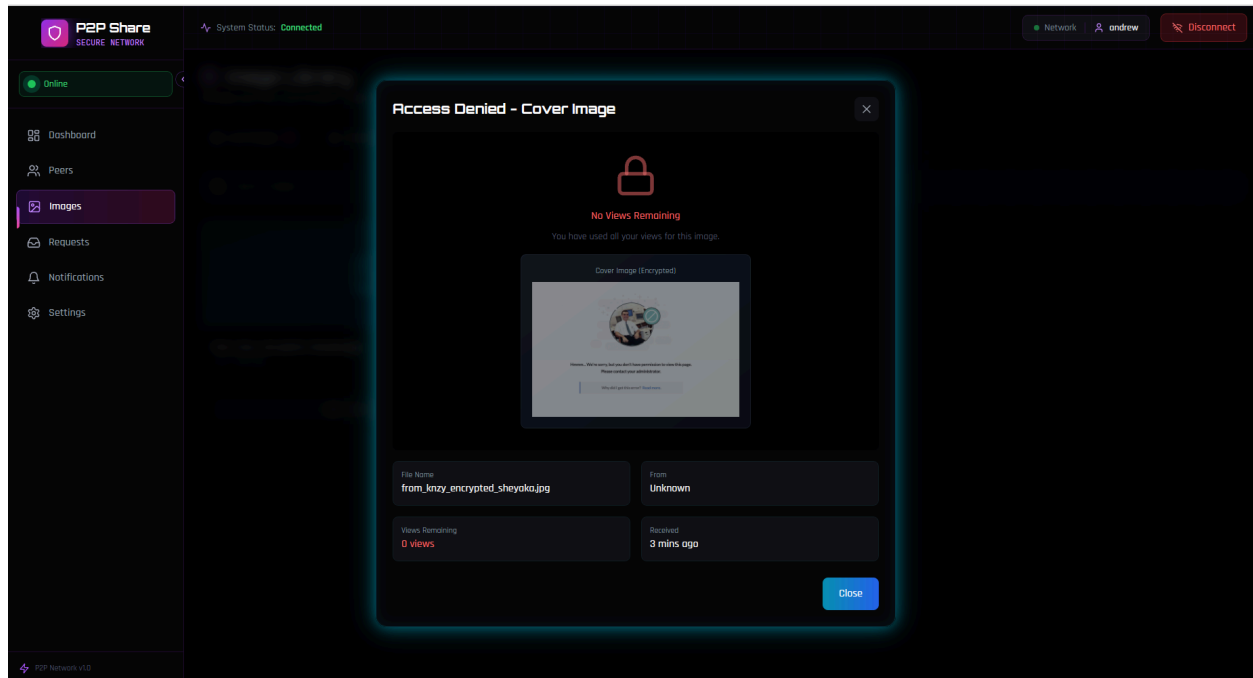


image-23 Now, client A returns online and tries to view the image, he got 0 views and access denied.

4. System Performance (From Phase 1 Report)

Our performance evaluation centers around the following key metrics:

- **Success Rate:** This metric measures the reliability of the system by showing the percentage of requests that were fully and correctly completed. We do so by dividing the successful requests counter by the total requests. Critically, a request is only counted as "successful" if it not only receives a response from the server but also passes the validate encrypted image function, which confirms the data is a valid PNG image.
- **Average Response Time (ms):** This metric measures the average performance, or latency, of the system only for successful operations. We do so by taking the total response time in ms (sum of all durations for successful requests) and dividing it by the successful_requests count. The duration for each request is measured from the moment the worker thread begins the task until it receives and validates the response as a proper PNG image, meaning this value represents the total time a user would wait for a correct result.

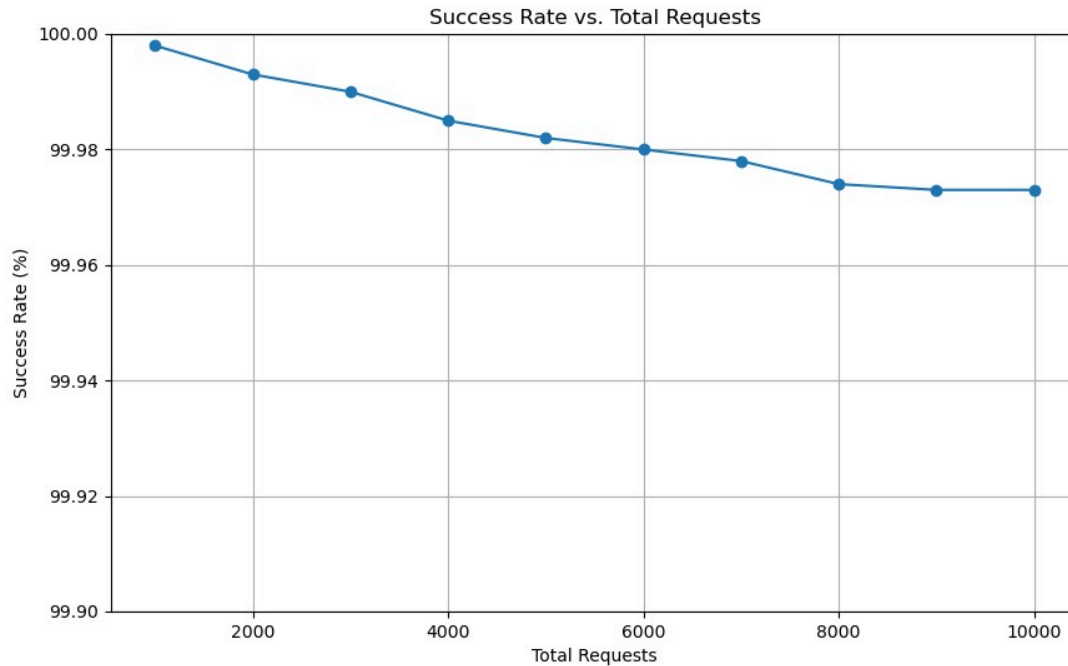
4.1 The Stress Testing Methodology

To evaluate our system's robustness and performance under pressure, we executed a multi-stage stress testing strategy. The initial phase consisted of incremental load tests, scaling from 1,000 up to 10,000 requests, to establish performance baselines. In one set of tests, we focused on fault tolerance, generating graphs that plotted the success rate and the average time per request against the increasing number of requests. A parallel analysis was conducted to measure the impact of our load balancing solution, comparing the average time per request in scenarios with and without load balancing active. Finally, to validate the entire system's stability in a production-like scenario, we conducted a large-scale peak test, subjecting the complete system with load balancing enabled to a total of 25,000 requests.

4.2 Analysis of System Success rate Under Load Balancing and Fault Conditions

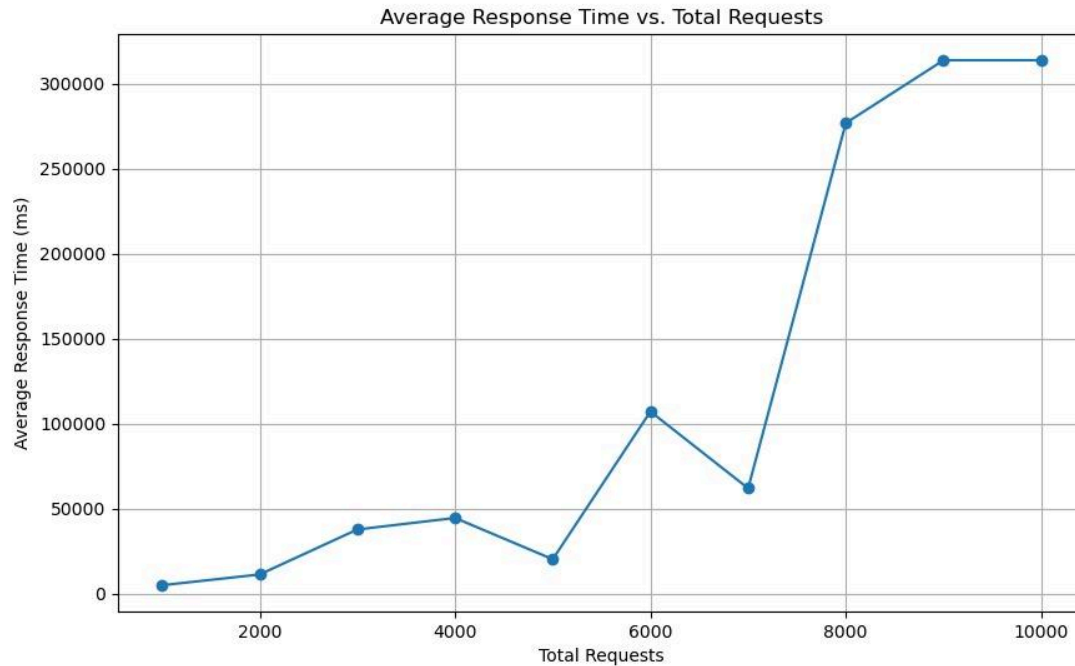
This graph, derived from our 3-server cluster stress test, highlights the system's robust reliability when subjected to both our load balancing strategy and continuous fault simulation. The data reveals two key characteristics. First, the system's overall performance is outstanding; the success rate consistently maintains a near-perfect level, staying above 99.97% even as the request load scales to 10,000. This demonstrates the resilience of our architecture, where both the Raft consensus protocol and the client's persistent retry mechanism successfully mitigates the vast majority of resulting failures.

The second characteristic is the minor, downward trend in success as load increases. This slight dip is not a flaw but rather the direct, observable consequence of our fault tolerance test in action. When our fault simulator forces a server offline (especially the leader), the cluster must initiate a new election. As the request volume increases, the statistical likelihood also increases that a client's request will exhaust all its retries precisely during one of these leaderless moments, resulting in a recorded failure. The fact that the success rate remains exceptionally high confirms our system's ability to recover quickly and efficiently.



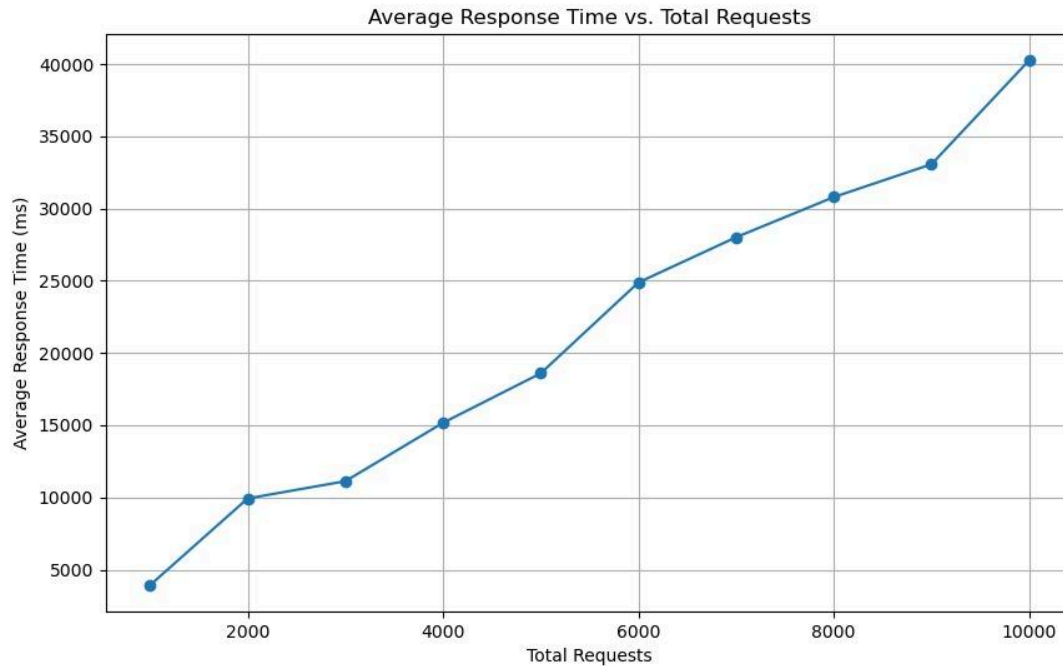
4.3 Analysis of System Average Response Time Under Load Balancing and Fault Conditions

This graph, plotting average response time against total requests, clearly illustrates the performance trade-offs of our fault-tolerant system when our load balancing and fault simulation are both active. The chart's erratic and spiky nature is the key finding. The dips in latency, such as those at 5000 and 7000 requests, demonstrate the success of our leader-delegation load balancing; when the cluster is stable, the leader effectively distributes work to less-busy follower nodes, optimizing performance. However, these efficiencies are punctuated by massive, non-linear latency spikes, most notably at 6000 requests and again from 8000 to 10000 requests. These spikes are the intended artifacts of our fault-tolerance test. They occur when our simulation "crashes" the cluster Leader, triggering a Raft leader election. During this election, the cluster has no leader, forcing all incoming client requests to queue and retry. The extreme average times (e.g., over 300,000 ms) are a direct measurement of this election-induced downtime combined with the subsequent backlog, as the newly elected leader must process the massive queue of requests that accumulated during the failure.



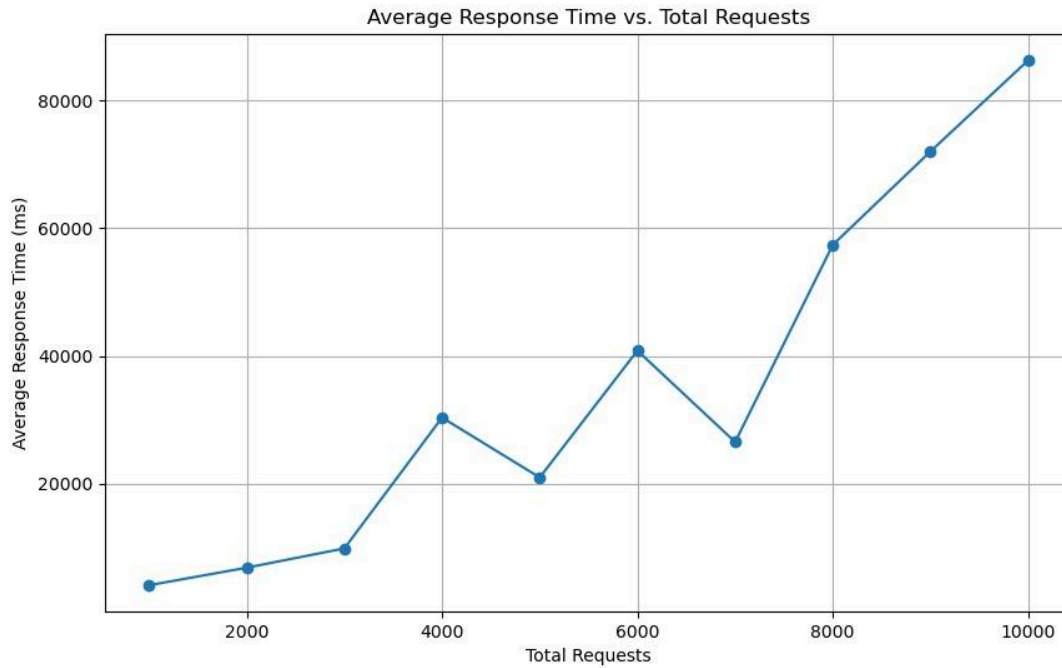
4.4 Analysis of System Average Response Time (Load Balancing Enabled, No Fault Simulation)

The graph illustrates the results of a stress test conducted on our cloud server cluster. The x-axis represents the total number of requests, while the y-axis shows the corresponding average response time in milliseconds. The data demonstrates a clear upward trend, indicating that as the request load increases, the average response time rises significantly. Starting at around 4,000 ms for 1,000 requests, the response time climbs steadily, reaching over 40,000 ms at 10,000 requests. This behavior is expected under higher loads, as system resources become increasingly saturated. Since the test was performed with load balancing enabled but without fault tolerance (i.e., no intentional server failures), the performance degradation observed can be attributed primarily to the growing request volume rather than server downtime. Overall, the results suggest that the load balancing mechanism performs effectively, successfully distributing requests across the three servers and maintaining stable operation even under increasing load. The system demonstrates solid scalability and consistent behavior, indicating that the Raft-based architecture can efficiently manage concurrent requests making it a reliable foundation for further optimization and future scalability improvements.



4.5 Analysis of System Average Response Time (No Load Balancing)

The graph presents the results of a stress test conducted on the same three-server cloud cluster running the Raft consensus algorithm, but this time without applying any load balancing mechanism. The x-axis represents the total number of requests, while the y-axis shows the average response time in milliseconds. The data reveals significant fluctuations in response time as the request load increases, with sharp rises and drops that indicate uneven request distribution and inconsistent performance across the servers. At lower loads, the response time remains relatively low, but as the number of requests grows beyond 4,000, the system begins to exhibit instability, with spikes exceeding 80,000 ms at the highest load. These variations suggest that, without load balancing, some servers become overloaded while others remain underutilized, leading to bottlenecks and degraded overall efficiency. Despite the absence of fault tolerance, the system continues to process requests, but its performance consistency and scalability are clearly limited. Overall, the results highlight the critical role of load balancing in maintaining stable response times and optimizing resource utilization across distributed servers.



4.6 Conclusion

Overall, the results of this project strongly validate the effectiveness and robustness of our system design. The integration of the Raft consensus algorithm, combined with our intelligent leader-delegation load balancing strategy, successfully delivers a highly available, fault-tolerant, and efficiently coordinated cloud-based P2P environment. Stress testing under both normal and fault conditions confirms that the system maintains exceptional reliability, achieving near-perfect success rates and demonstrating rapid recovery from simulated server failures. Even under heavy request loads, the architecture continues to operate with stability and consistency, proving that the chosen design not only meets but exceeds the core requirements of scalability, reliability, and performance. These findings affirm that our design choices—particularly the adoption of Raft, TCP-based communication, and distributed load coordination—provide a strong, future-ready foundation for secure and scalable image sharing in cloud environments.

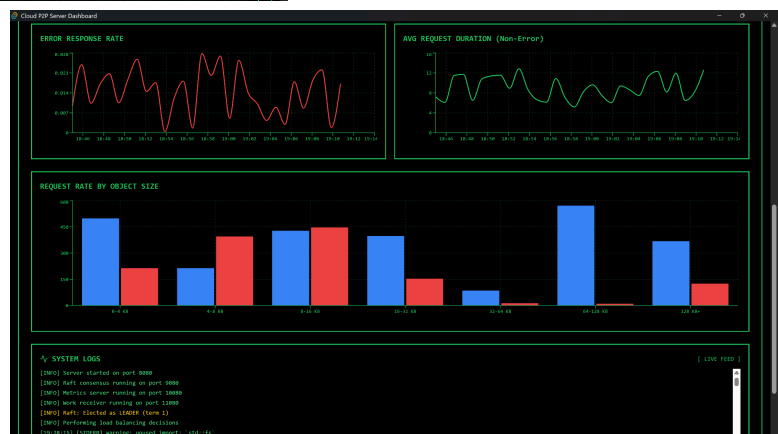
5. Future Improvements & System Evolution

A significant portion of development was dedicated to a high-fidelity **Cloud P2P Server Control Terminal**. Although currently operating as a standalone prototype, full integration of this GUI with the Rust backend is the primary next step.

The dashboard functionality demonstrates promising capabilities for system observability:

- ❖ **Real-Time Cluster Visualization:** The GUI visualizes the **Raft Consensus Protocol** states, identifying which nodes are Leaders versus Followers (e.g., `server_1` as LEADER).
- ❖ **Live Metrics Streaming:** It displays critical telemetry such as "Error Response Rate," "Average Request Duration," and traffic distribution by object size.
- ❖ **Fault Injection & Control:** The interface includes controls to "Simulate Failure," "Stop All," or "Start All" nodes. Integrating these buttons to trigger actual events in the Rust backend would allow for rigorous stress testing of the system's fault tolerance and leader election logic.

We have attempted to develop a GUI for the servers as an extra feature, however, time was not in our favor. It was a functional GUI in terms of starting servers, stopping them, and simulating failure. It also correctly displayed logs and number of connections/clients. We wanted to edit the graphs and figures that it dynamically updates, but did not have the time to. We used mock data for the graphs. We might be working on it more though to publish it publicly on github and if we do, we will make sure to share the link! :)



6. References

1. M. Gholipour, M. S. Kordafshari, M. Jahanshahi and A. M. Rahmani, "A New Approach for Election Algorithm in Distributed Systems," 2009 Second International Conference on Communication Theory, Reliability, and Quality of Service, Colmar, France, 2009, pp. 70-74, doi: 10.1109/CTRQ.2009.32.
2. Coulouris, George, et al. *Distributed Systems: Concepts and Design*. 5th ed., Addison-Wesley, 2012.